



POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



Programming with the 8086/8088 microprocessor

Faculty: Computer science – IT1

Instructor: Viet Hoang Pham

Email: vietph@ptit.edu.vn

Phone: 0945296388

- Internal Architecture of 8086/8088
- Instruction Set
- Introduction to Assembly Language
- Assembly Language Framework
 - Some Control Structures and Examples
 - Subroutines and Marco
- Introduction to emu8086 Simulation Software
 - Some Illustrative Examples
 - Introduction to Virtual Devices - Simulation of Traffic Lights, Thermometers, and Stoves

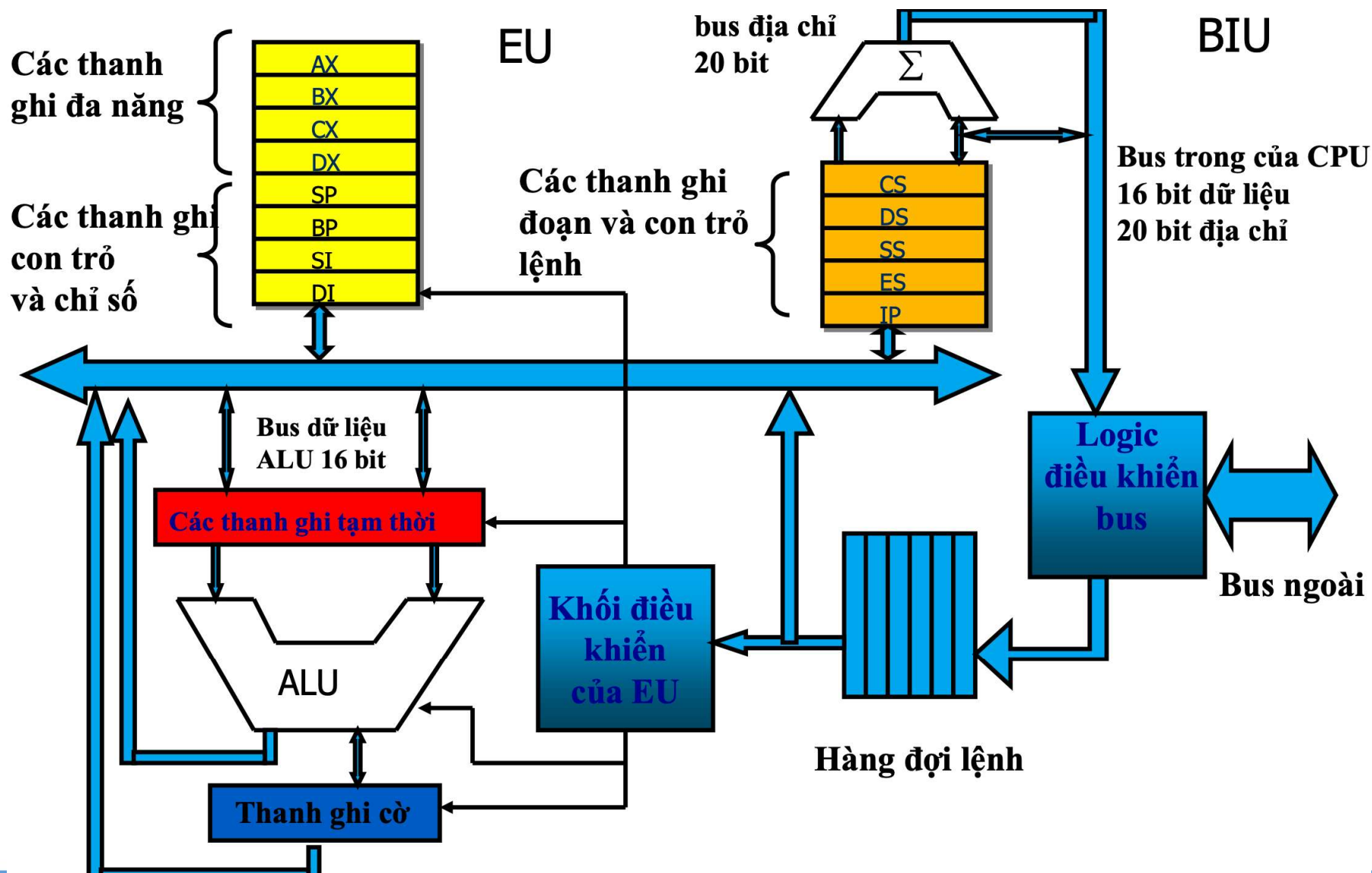
8088



8086



❖ Block diagram of the 8086/8088



❖ Microprocessor Components

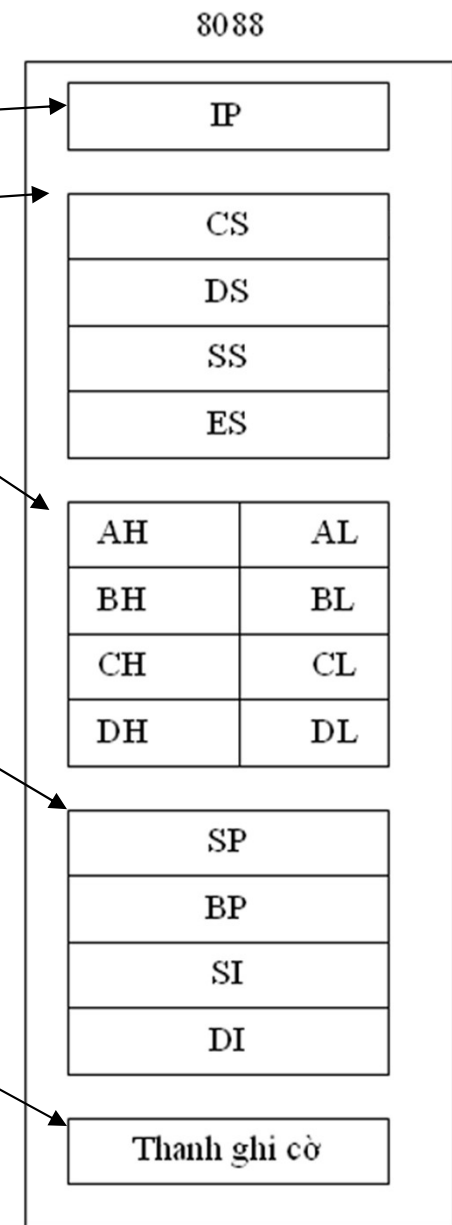
- Execution Unit (EU):
 - Function: The EU receives commands and data from the BIU for processing. The processing results are transferred to memory or I/O devices via the BIU.
 - Blocks:
 - ALU
 - CU
 - 8 16-bit registers: AX, BX, CX, DX, SP, BP, SI, DI
 - Flag register FR
- Internal Bus: links BIU and EU:
 - 16-bit A-BUS in 8088
 - 16-bit ALU-BUS in 8086

❖ Microprocessor Components

- Bus Interface Unit (BIU)
 - System bus control: outputs addresses to the bus and exchanges data with the bus:
 - Outputs addresses
 - Reads instruction code from memory
 - Reads/writes data to/from memory or input/output ports
 - Blocks:
 - Adder for address calculation
 - 4 16-bit segment registers: CS, DS, SS, ES
 - 16-bit program counter/instruction pointer (PC/IP)
 - Queue IQ instruction (4 bytes in 8088 and 6 bytes in 8086)
 - Bus control logic

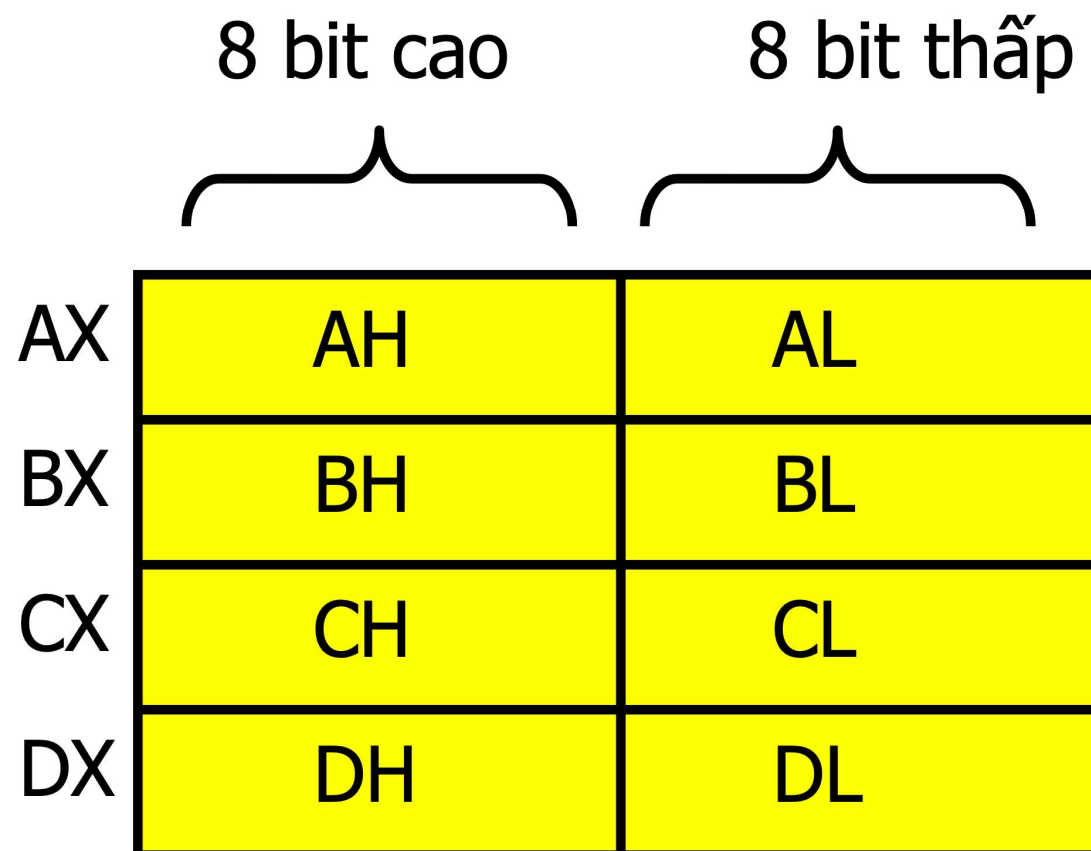
❖ General-purpose registers

- Instruction Pointer
- Segment registers
- 4 x 16-bit registers or 8 x 8-bit registers
- Pointer and Index Registers
- Flag Registers



❖ General-purpose registers

- 4 x 16-bit registers or 8 x 8-bit registers



❖ General-purpose registers

■ 4 x 16-bit registers:

- AX (accumulator): Sum register, usually used to store results
- BX (base): Base register, usually used to store memory addresses
- CX (count): Count register, usually used as a counter for loop instructions, containing the number of shifts or rotations in shift and rotate instructions
- DX (data): Data register, along with AX, contains data in 16-bit multiplication/division calculations or the port address in direct input/output (IN/OUT) instructions

Or 8 x 8-bit registers: AH AL, BH, BL, CH, CL, DH, DL

❖ General-purpose registers

■ Pointer and Index Registers

SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index

❖ General-purpose registers

■ Pointer and Index Registers

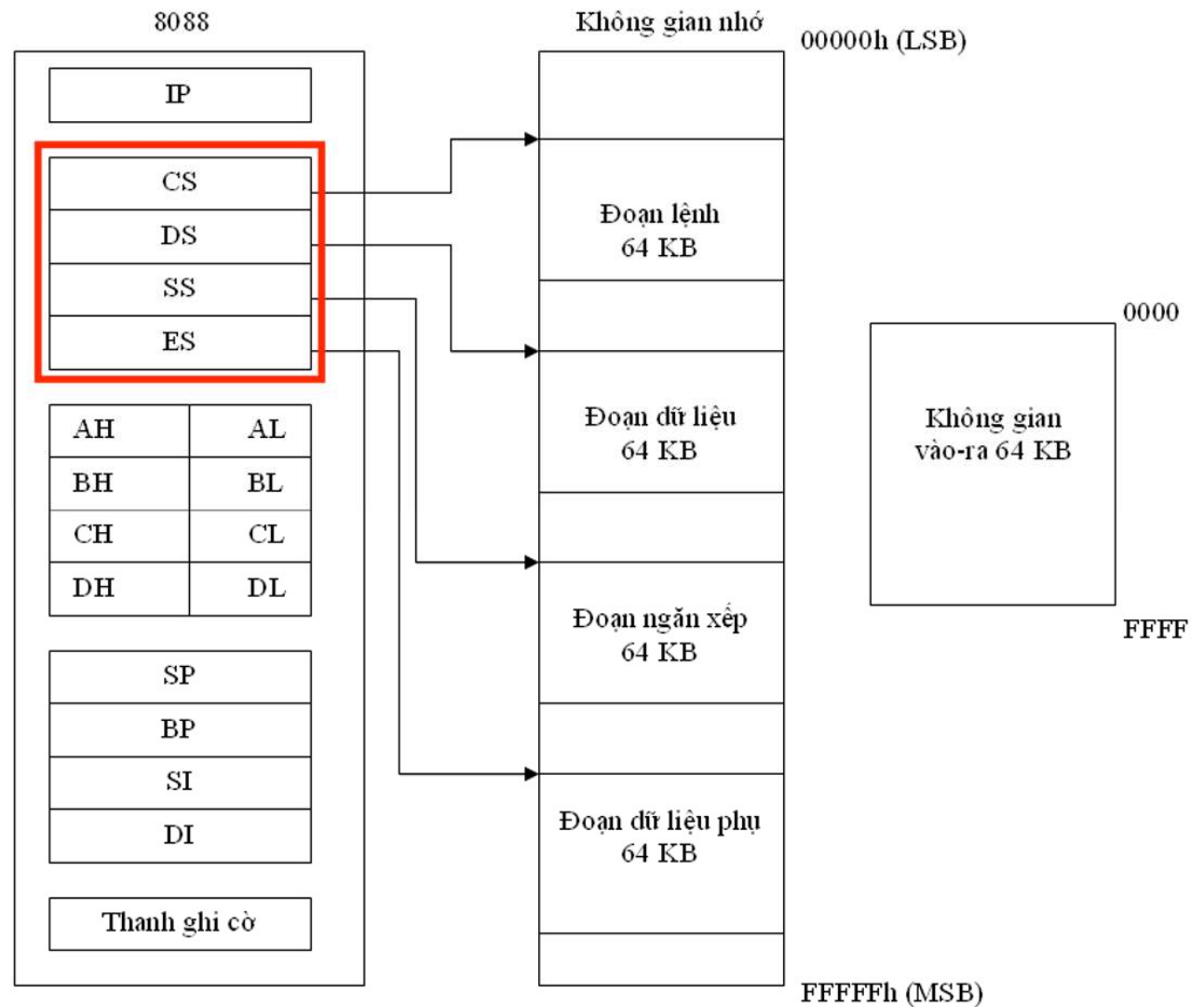
- SP (Stack Pointer): Stack pointer. SP always contains the address of the top of the stack SS:SP
- BP (Base Pointer): Base pointer, contains the address of the data in the stack segment SS or other segments SS:BP
- SI (Source Index): Source index register. SI is often used to contain the address of the source memory location in the data segment DS, as in string instructions in data transfer operations DS:SI
- DI (Destination Index): Destination index register. DI is often used to contain the address of the destination memory location in the data segment DS in data transfer operations DS:DI

SI and DI can be used as general-purpose registers

80386 and later 32-bit: EIP, EBP, ESP, EDI, ESI

❖ General-purpose registers

■ Segment registers:



❖ General-purpose registers

■ Segment registers:

- CS (Code Segment): The code segment register. CS contains the starting address of the code segment.
- DS (Data Segment): The data segment register. DS contains the starting address of the data segment.
- SS (Stack Segment): The stack segment register. SS contains the starting address of the stack segment.
- ES (Extra Segment): The extra data segment register. ES contains the starting address of the extra data segment.

❖ General-purpose registers

■ Instruction Pointer (IP):

- Instruction pointer (also called program counter - PC) always contains the address of the next instruction to be executed;
 - The IP register (instruction pointer) contains the offset address of the next instruction.
 - CS:IP contains the logical address of that next instruction.

❖ General-purpose registers

■ Flag Registers (FR):

- FR (Flag Register) or SR (Status Register): Flag register or status register.
- Status flag: The bits of FR store the states of the operations performed by the ALU.
- Control flag: the state of the control signal.



❖ General-purpose registers

- Flag Registers (FR) - Status flags:
 - CF (Carry):
 - Carry flag. $CF = 1$ indicates a carry operation;
 - $CF = 0$ indicates a carry operation. If the addition operation carries from the leading bit or the subtraction operation borrows from the leading bit, CF is set (indicates an overflow with an unsigned integer).
 - AF (Auxiliary): Auxiliary carry flag.
 - $AF = 1$ indicates an auxiliary carry;
 - $AF = 0$ indicates no auxiliary carry. If the addition operation carries from bit 3 to bit 4 or the subtraction operation borrows from bit 3 to bit 4, AF is set

❖ General-purpose registers

■ Flag Registers (FR) - Status flags:

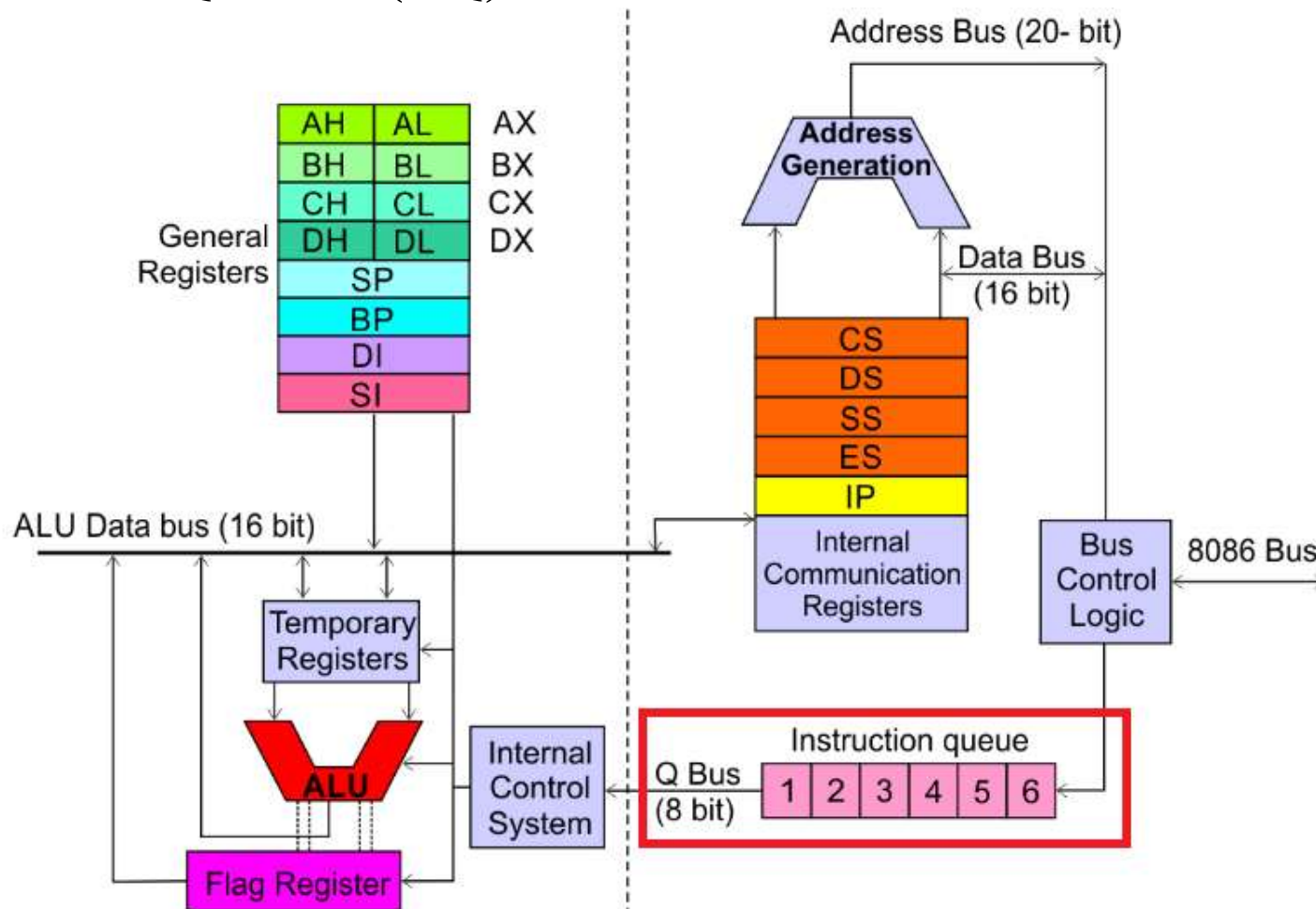
- PF (Parity): Parity flag. $PF = 1$ when the sum of the 1 bits in the result is odd and $PF = 0$ when the sum of the 1 bits in the result is even.
- OF (Overflow): Overflow flag. $OF = 1$ when the result overflows. If adding two numbers with the same sign results in a sign opposite, OF is set (indicates an overflow with a signed integer).
- ZF (Zero): Zero flag. $ZF = 1$ when the result is 0; otherwise, $ZF = 0$.
- SF (Sign): Sign flag. $SF = 1$ when the result is negative.

❖ General-purpose registers

- Flag Registers (FR) - Control Flags:
 - DF (Direction): Direction flag, indicates the direction of address increment/decrement for data transfer instructions.
 - DF = 0 is an increment address.
 - DF = 1 is a decrement address.
 - TF (Trap/Trace): Trap/trace flag, used when debugging a program.
 - TF = 1 means the CPU is in instruction-by-instruction mode (debugging mode).
 - IF (Interrupt): Interrupt flag.
 - If IF = 1, the microprocessor enables interrupts with interrupt requests sent to the INTR (Interrupt Request) signal pin of the microprocessor.
 - If IF = 0, interrupts are disabled.

❖ General-purpose registers

■ Instruction Queue (IQ)

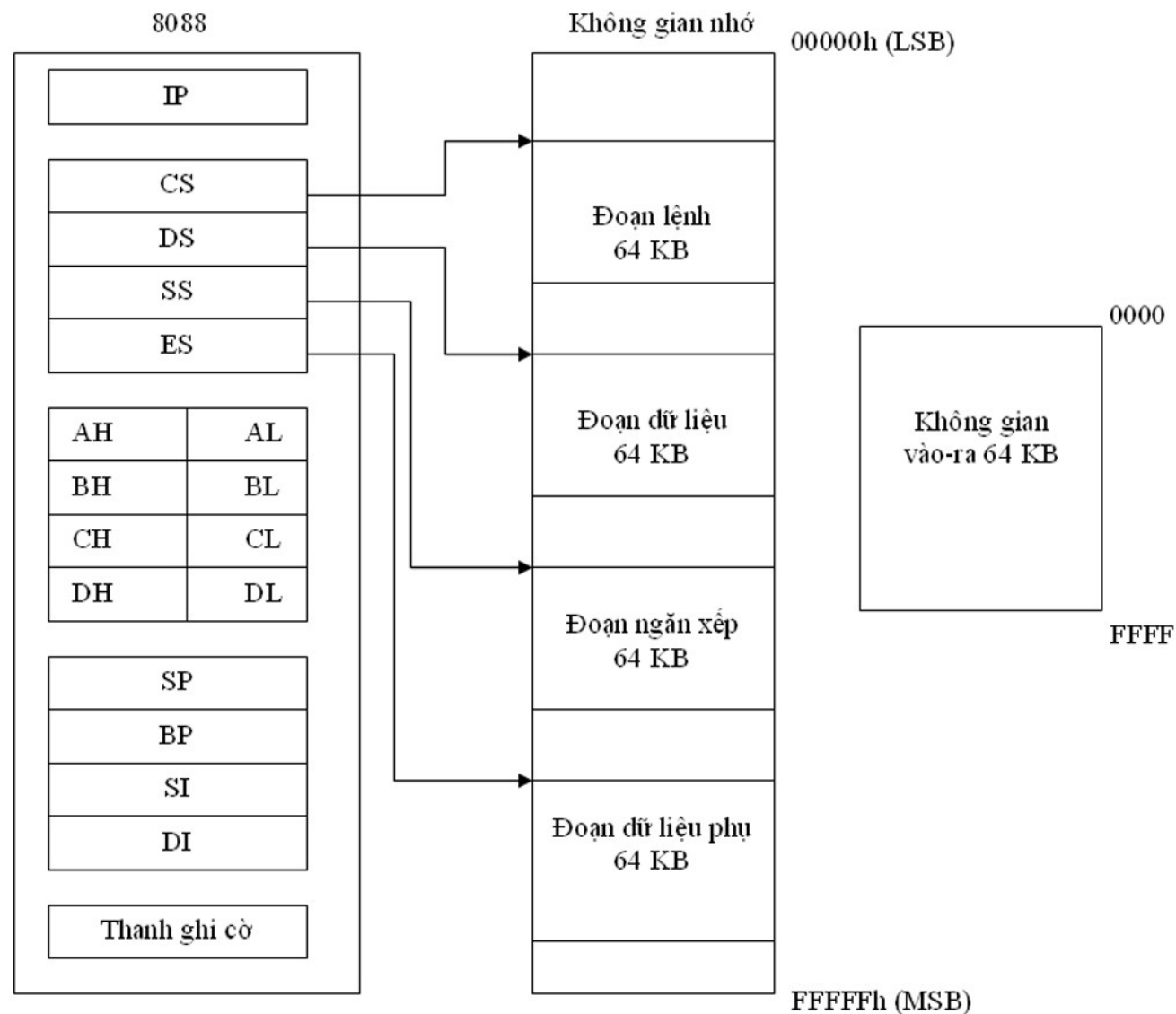


❖ General-purpose registers

■ Instruction Queue (IQ)

- Contains read instructions from memory for the EU to execute.
- In the 8088, the IQ has 4 bytes, while in the 8086, the IQ has 6 bytes.
- The IQ is a crucial component of the instruction pipeline mechanism that helps speed up instruction processing.

❖ Characteristics



❖ The memory organization of the 1 Mbyte:

8088/8086 microcontroller uses 20 bits for memory addressing:

- The total maximum addressable memory capacity is 2^{20} bytes = 1MB;
- Physical addresses are numbered from 00000h to FFFFFh.
- The 8088's memory space is divided into memory segments with a capacity of 64 KB = 2^{16} bytes.
- Segment:Offset

$$\text{Physical address} = \text{Segment} \times 16 + \text{Offset}$$

- The segments can overlap.

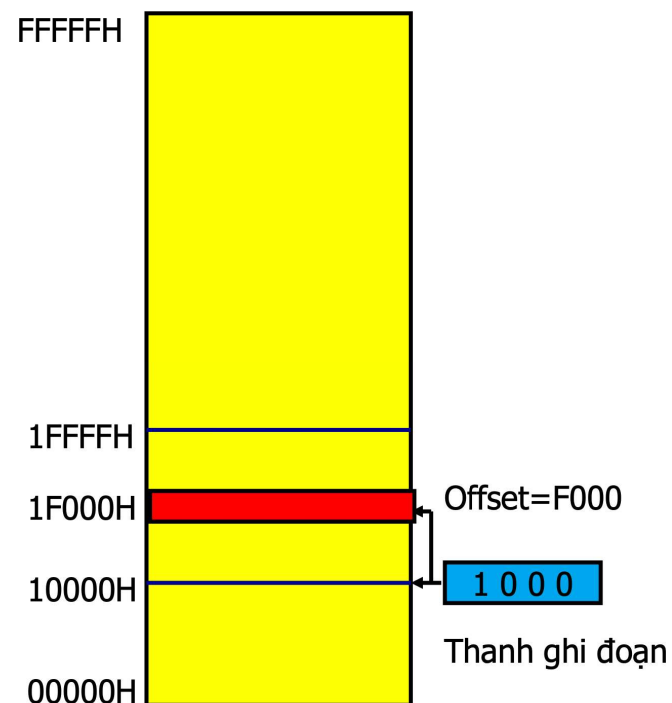
1000:0010 and 1001:0000 can create the same physical address.

❖ The memory organization of the 1 Mbyte:

■ Memory segment:

- The starting address of each memory segment is divisible by 16 → The starting address of each segment is in the form xxxx0h (x: any hexadecimal character)
- The memory segment address only needs to store 4 hexadecimal numbers (16 high bits)
 - Segment address: xxxxh
 - Physical address of the beginning of the segment: xxxx0h
 - Physical address of the last memory location of the segment: xxxx0h + FFFFh

Example: If the segment address is 1234h, then the physical address of the beginning of that memory segment is 12340h



❖ The memory organization of the 1 Mbyte:

- Memory location within a segment:
 - Segment addresses are managed by segment registers.
 - Offset addresses are managed by IP, BX, BP, SP, SI, and DI registers.
 - Offset address: offset
 - First location: offset: 0000
 - Last location: offset: FFFF
- Logical address in the 8086 microcontroller is in the form: Segment (16-bit segment address) : offset (16 bits)
- Physical address (20 bits) = Segment address X 10h + offset
- The 20-bit physical address of a memory location is determined by adding the 16-bit segment address shifted left by 4 bits (multiplied by 16) and the 16-bit offset address.

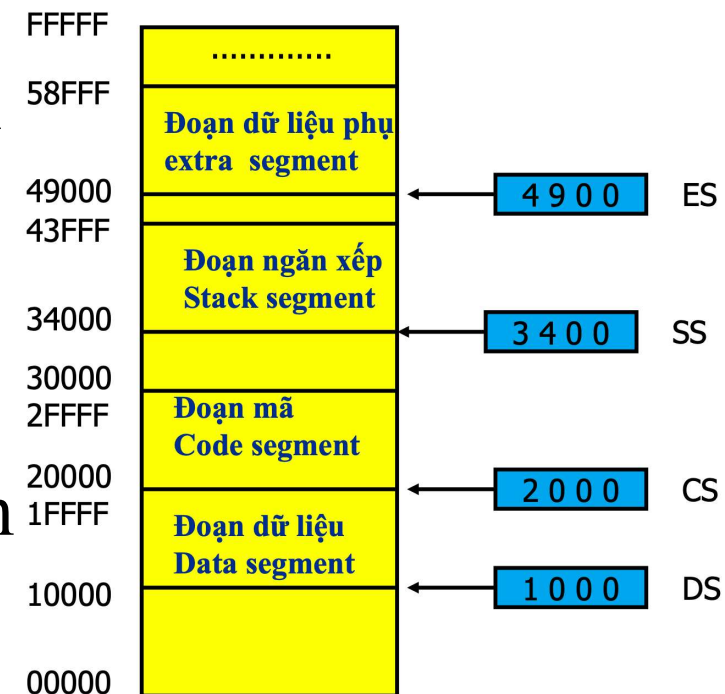
❖ The memory organization of the 1 Mbyte:

- For example, CS:IP indicates the address of the instruction to be executed in the code.

- If CS=F000h and IP=FFF0h, then CS:IP corresponds to

$$F000h * 16 + FFF0h = F0000h + FFF0h = FFFF0h.$$

- The programmer only programs with logical addresses; the conversion to physical addresses is done by the microprocessor.



❖ Offset address

- Instruction pointer IP: contains the address of the next instruction in the corresponding instruction segment CS CS:IP
- Base pointer BP: contains the address of the data in the stack segment SS or other corresponding segments SS:BP
- Stack pointer SP: contains the current address of the top of the stack SS:SP
- Source index SI: contains the address of the source data in the data segment DS in the corresponding instruction sequence DS:SI
- Destination index DI: contains the address of the destination data in the data segment DS in the corresponding instruction sequence DS:DI

SI and DI can be used as general-purpose registers

80386 and later 32-bit: EIP, EBP, ESP, EDI, ESI

❖ Instructions and How Instructions Are Encoded

- What is an instruction? It is a binary word:
 - Instructions are stored in memory
 - Instructions are loaded into the CPU for execution
 - Each instruction has a specific task
 - Common instruction groups: data transport, program control, computation, etc.
- The main phases of instruction execution:
 - Instruction Fetch (IF)
 - Instruction Decode (ID)
 - Instruction Execution (EX)

❖ Instructions and How Instructions Are Encoded

■ Instruction cycle:

- This is the time it takes for the CPU to complete one instruction.
- Each phase of an instruction consists of a number of machine cycles.
- Each machine cycle consists of a number of clock cycles.

❖ Instructions and How Instructions Are Encoded

- An instruction cycle can include:
 - Instruction read cycle
 - Memory (data) read cycle
 - Memory (data) write cycle
 - I/O (data) read cycle
 - I/O (data) write cycle
 - Interrupt acknowledgment cycle
 - Idle bus

❖ Instructions and How Instructions Are Encoded

- Instruction encoding method:
 - The general form of an instruction consists of two components: the instruction code and the addresses of the operands.
 - Instruction word length: 8, 16, 24, 32, and 64 bits.
 - 8086/8088 instructions can be 1-6 bytes long.

Opcode	Operands
---------------	-----------------

Mã lệnh

Các toán hạng

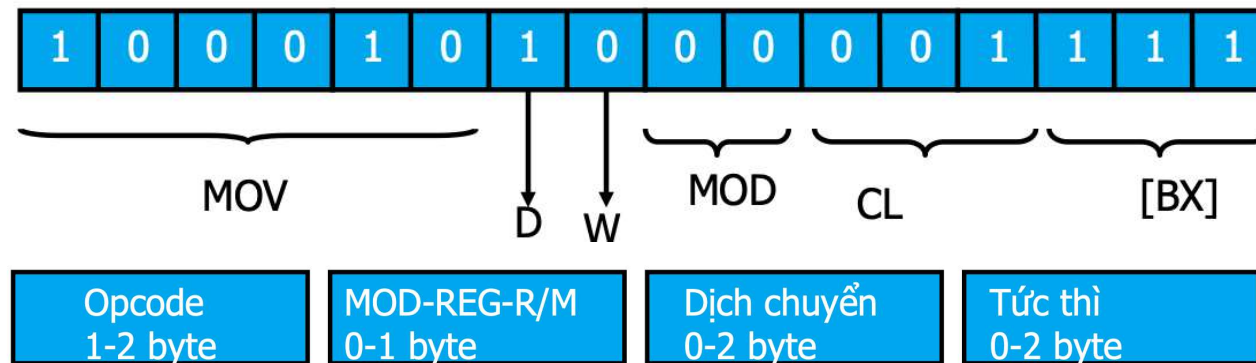
Mã lệnh	Đích, Gốc
----------------	------------------

MOV	AX, 100
------------	----------------

$AX \leftarrow 100$

❖ Instructions and How Instructions Are Encoded

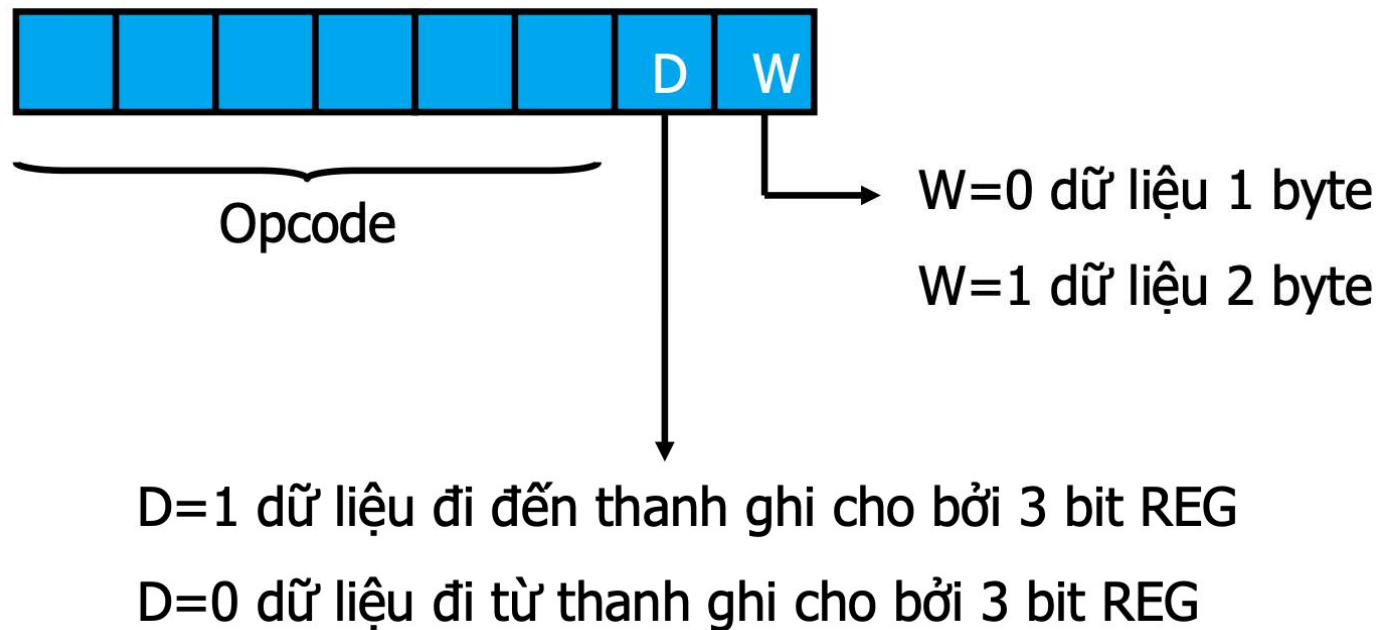
■ Instruction encoding method:



- Opcode: 6-bit instruction code
- D: direction bit, indicates the direction of data transfer;
 - D = 1: data goes to the register specified by the 3-bit REG
 - D = 0: data goes out of the register specified by the 3-bit REG
- W: bit indicating operand width;
 - W=0: 1-byte (8-bit) operand;
 - W=1: 2-byte (16-bit) operand

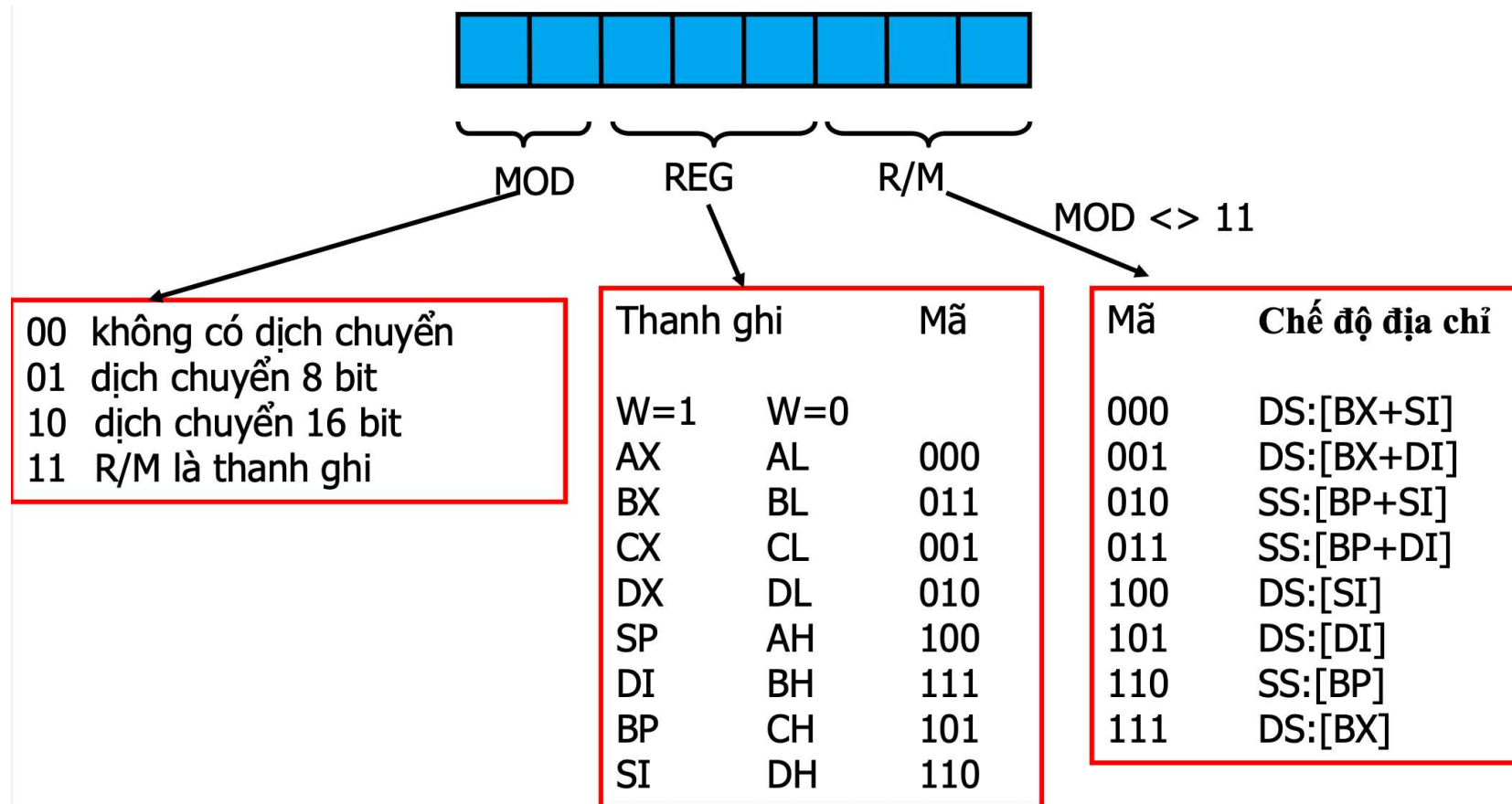
❖ Instructions and How Instructions Are Encoded

- Instruction encoding method:



❖ Instructions and How Instructions Are Encoded

■ Instruction encoding method:



❖ Instructions and How Instructions Are Encoded

■ Encoding registers

Các thanh ghi đoạn	Mã thanh ghi
CS	01
DS	11
ES	00
SS	10

Các thanh ghi		Mã thanh ghi
W=1	W=0	
AX	AL	000
BX	BL	011
CX	CL	001
DX	DL	010
SP	AH	100
DI	BH	111
BP	CH	101
SI	DH	110

❖ Instructions and How Instructions Are Encoded

■ Encoding registers

MOD R/M	00	01	10	11	
				W=0	W=1
000	[BX]+[SI]	[BX]+[SI]+d8	[BX]+[SI]+d16	AL	AX
001	[BX]+[DI]	[BX]+[DI]+d8	[BX]+[DI]+d16	CL	CX
010	[BP]+[SI]	[BP]+[SI]+d8	[BP]+[SI]+d16	DL	DX
011	[BP]+[DI]	[BP]+[DI]+d8	[BP]+[DI]+d16	BL	BX
100	[SI]	[SI]+d8	[SI]+d16	AH	SP
101	[DI]	[DI]+d8	[DI]+d16	CH	BP
110	d16	[BP]+d8	[BP]+d16	DH	SI
111	[BX]	[BX]+d8	[BX]+d16	BH	DI

<

Các chế độ bộ nhớ

>

Các chế độ
thanh ghi

Ghi chú:

- d8: khoảng dịch chuyển, 8 bit
- d16: khoảng dịch chuyển, 16 bit

❖ Instructions and How Instructions Are Encoded

■ Instruction encoding method:

Opcode 1-2 byte	MOD-REG-R/M 0-1 byte	Dịch chuyển 0-2 byte	Tức thì 0-2 byte
--------------------	-------------------------	-------------------------	---------------------

- Displacement is the offset value used when accessing memory. It's added to the base address to calculate the effective address (EA).

Example:

–MOV AX, [BX + 5] → 5 is an 8-bit displacement.

–MOV AX, [BP + 1234h] → 1234h is a 16-bit displacement.

$EA = \text{Base} + \text{Index} + \text{Displacement}$

- Immediate is data written directly in the instruction.

Example: MOV AX, 1234h, ADD AL, 05h → 1234h and 05h are the immediate data.

Depends on: W bit (8 bits or 16 bits) and Instruction type

- ❖ Addressing mode is how the CPU organizes and retrieves data for operands when executing instructions. A microprocessor can have multiple addressing modes.
- ❖ The 8086/8088 microprocessor has 7 addressing modes:
 1. Register Addressing Mode
 2. Immediate Addressing Mode
 3. Direct Addressing Mode
 4. Register Indirect Addressing Mode
 5. Based Plus Displacement Addressing Mode
 6. Indexed Plus Displacement Addressing Mode
 7. Based Indexed Plus Displacement Addressing Mode

❖ Some conventions regarding 8086/8088 instructions:

- 8086/88 assembly is not case-sensitive.
- 100: Decimal constant.
- 80H, 0F9H: Hexadecimal constants starting with a letter (A-F) have a 0 added to the beginning and the symbol H (Hexadecimal) added to the end.
- 0111B, 1000B: Binary constants have the symbol B (Binary) added to the end.
- [AL]: Memory location whose address is stored in the AL register.
- [100]: Memory location with address 100.
- [BX+100]: Memory location whose address is stored in the memory location with address $BX + 100$ in the data segment.

❖ Register Addressing Mode:

- Register mode uses internal CPU registers as operands to hold the data to be manipulated.
- Both the source and destination operands are registers.
- High instruction execution speed.
- Examples:
 - MOV BX, DX; $BX \leftarrow DX$
 - MOV DS, AX; $DS \leftarrow AX$
 - ADD AL, DL; $AL \leftarrow AL + DL$
 - MOV AL, BX; invalid because the two registers are different sizes
 - MOV ES, DS; invalid (segment to segment)
 - MOV CS, AX; invalid because CS is not used as the destination register

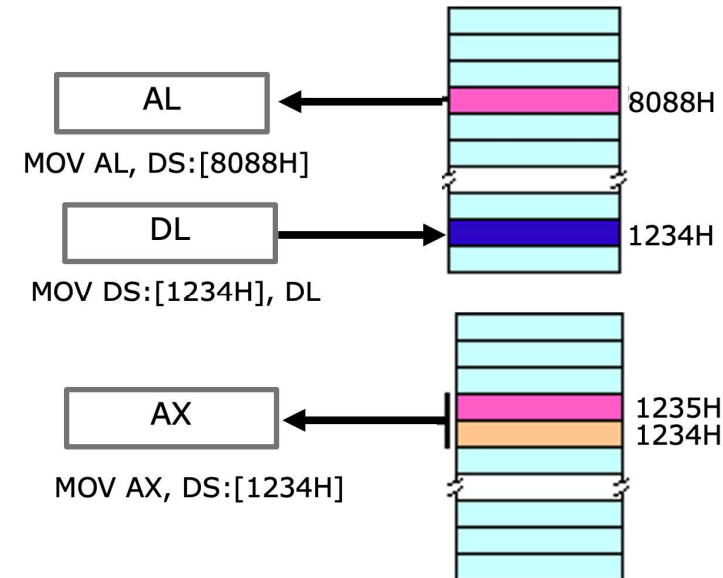
❖ Immediate addressing mode:

- The destination operand is a register or a memory location
- The source operand is a constant
- Used to load constants into registers (except segment registers and flag registers) or into memory locations within the DS data segment
- Examples:
 - MOV CL, 200; $CL \leftarrow 200$
 - MOV AX, 0ff0h; $AX \leftarrow 0ff0h$
 - MOV [BX], 200; Moves 200 to the memory location with address DS:BX
 - MOV AL, 'A'; Copies the ASCII code of A to the AL register
 - MOV DS, 0FF0H; Invalid

❖ Direct addressing mode:

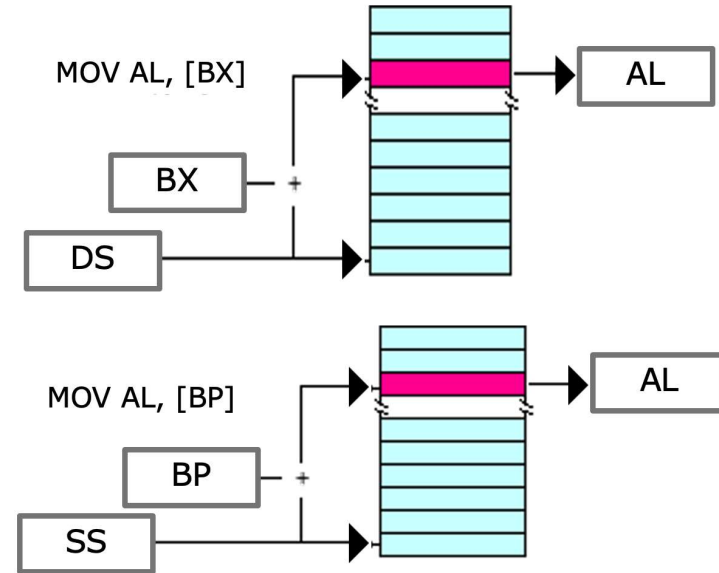
- One operand is a constant representing the offset address of the memory location.
- The other operand can be a register (it cannot be a memory location).
- Examples:

- `MOV AX, [2000H];` $AX \leftarrow M[DS:2000H]$
- `MOV [1050H], BX;` Write the BX content to memory location DS:1050H
- `ADD AL, [3000H];` $AL \leftarrow AL + \text{content at } DS:3000H$
- `MOV [1000H], [2000H];` invalid



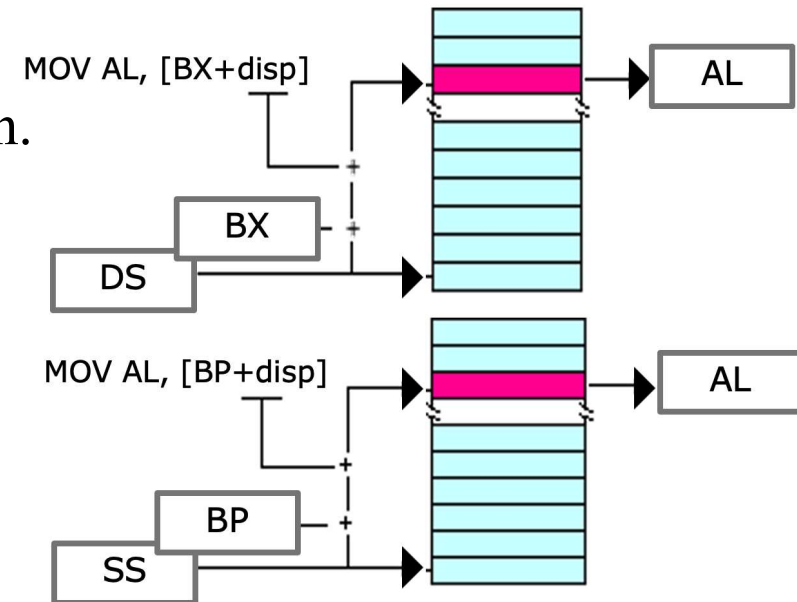
❖ Registers indirect addressing mode :

- One operand is a register containing the offset address of a memory location.
- The other operand can be a register.
- Examples:
 - `MOV AL, [BX]; AL  $\leftarrow$  [DS:BX]`
 - `MOV AL, [BP]; AL  $\leftarrow$  [SS:BP]`
 - `MOV [DI], AX; Copy the contents of AX to two consecutive memory locations DS:DI and DS:(DI+1)`



❖ Base Plus Displacement Addressing Mode:

- One operand is the address of a memory location.
 - The memory location address is created by a base register such as BX (DS segment) or BP (SS segment) and a constant.
 - The constant is the displacement value used to calculate the effective address of the operands in the DS and SS memory areas.
- The remaining operand can be a register (not a memory location).
- Examples:
 - `MOV AL, [BX+100];` $AL \leftarrow [DS: BX+100]$
 - `MOV AL, [BP+200];` $AL \leftarrow [SS: BP+200]$
 - `MOV AL, [BP]+200;` Alternative way of writing the above instruction.

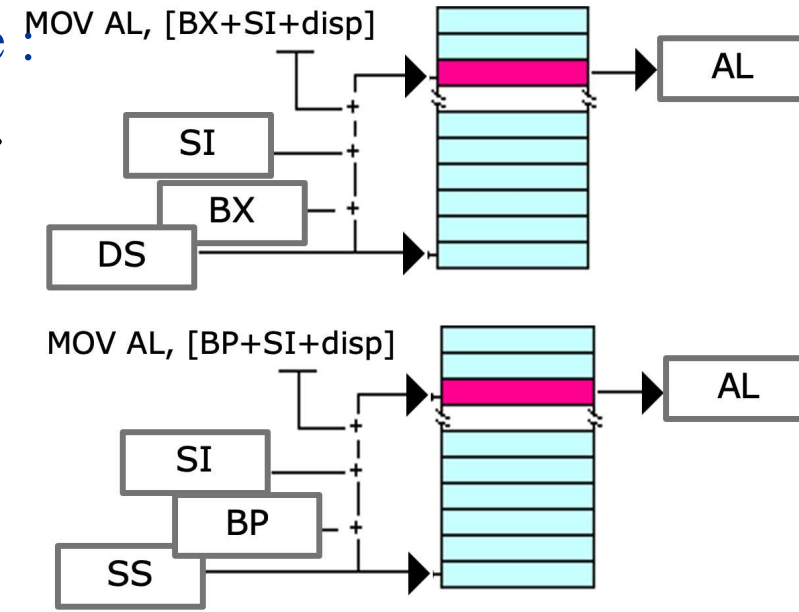


❖ Indexed Plus Displacement Addressing Mode:

- One operand is the address of a memory location.
 - The memory location address creates the base register SI or DI and a constant.
 - The constant represents the displacement values used to calculate the effective address of the operands in the DS memory areas.
- The remaining operand can be a register (it cannot be a memory location).
- Examples:
 - `MOV AL, [SI+100]; AL ← [DS: SI+100]`
 - `MOV AL, [DI+200]; AX ← [DS: DI+200]`
 - `MOV AX, [SI]+10;` Copy the contents of two consecutive memory locations with addresses DS:SI+10 and DS:SI+11 into AX
 - `MOV AX, [SI+10];` Another way to write the above instruction

❖ Based Indexed Plus Displacement Addressing Mode :

- One operand is the address of a memory location.
 - The memory location address is created by the registers BX+SI/DI (DS segment) or BP+SI/DI (SS segment) and a constant.
 - The constant represents the displacement values to calculate the effective address of the operands in the DS and SS memory areas.
- The remaining operand can be a register (it cannot be a memory location).
 - `MOV AL, [BX+SI+100];` $AL \leftarrow [DS:BX+SI+100]$
 - `MOV AX, [BX] [SI]+8;` Copy the contents of two consecutive memory locations with addresses `DS:BX+SI+8` and `DS:BX+SI+9` into AX.
 - `MOV AX, [BX+SI+8];` Another way to write the above instruction.



❖ Implicit mapping in addressing modes

Chế độ địa chỉ	Toán hạng	Đoạn ngầm định
Thanh ghi	Reg	
Tức thì	Data	
Trực tiếp	[offset]	DS
Gián tiếp qua thanh ghi	[BX]	DS
	[BP]	SS
	[SI]	DS
	[DI]	DS
Tương đối cơ sở	[BX] + Disp	DS
	[BP] + Disp	SS
Tương đối chỉ số	[SI] + Disp	DS
	[DI] + Disp	DS
Tương đối chỉ số cơ sở	[BX] + [SI] + Disp	DS
	[BX] + [DI] + Disp	DS
	[BP] + [SI] + Disp	SS
	[BP] + [DI] + Disp	SS

❖ Implicit relationships between segment registers and instruction registers:

Thanh ghi đoạn	CS	DS	ES	SS
Thanh ghi lệnh	IP	SI, DI, BX	DI	SP, BP

■ Implicit addresses:

- `MOV AL, [BX]; AL  $\leftarrow$  [DS:BX]`
- `MOV [SI+300], AH; [DS:SI+300]  $\leftarrow$  AH`

■ Explicit (full) addresses:

- `MOV AL, ES:[BX]; AL  $\leftarrow$  [ES:BX]`
- `MOV SS:[SI+300], AH; [SS:SI+300]  $\leftarrow$  AH`

❖ Microprocessor Instruction Set Classification: Complex Instruction Set (CISC) and Reduced Instruction Set (RISC)

- CISC (Complex Instruction Set Computers)
 - Supports a rich instruction set → reduces the amount of program code
 - Large instruction set → difficult to optimize for compiler
 - Instructions have different lengths and execution times → reduces the performance of the instruction pipeline mechanism
- RISC (Reduced Instruction Set Computers)
 - Minimal instruction set: number of instructions, addressing modes, instruction format, and execution time
 - Increases the performance of the instruction pipeline mechanism
 - Easy to optimize in compiler
 - Programs are usually long, require a lot of memory, and increase memory access time

❖ Classification of instruction sets for CISC microprocessors:

1. Data Transport
2. Integer and Logic Arithmetic
3. Shift and Rotate
4. Control Transfer
5. Bit Handling
6. System Control
7. Floating-Point Operations
8. Special Function Unit Instructions

❖ Data Transfer Instructions:

- Transports data between:
 - Register – register;
 - Register – memory location;
 - Register – input/output device.
- Instructions:
 - MOV, XCHG, POP, PUSH, POPF, PUSHF, IN, OUT
 - LODSB, LODSW, STOSB, STOSW and string shift instructions MOVS, MOVSB, MOVSW.

Data transport instructions do not affect the status flags of the flag register.

❖ Data Transfer Instructions:

MOV = Move:

	7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0
Register/Memory to/from Register	1 0 0 0 1 0 d w	mod reg r/m		
Immediate to Register/Memory	1 1 0 0 0 1 1 w	mod 0 0 0 r/m	data	data if w = 1
Immediate to Register	1 0 1 1 w reg	data	data if w = 1	
Memory to Accumulator	1 0 1 0 0 0 0 w	addr-low	addr-high	
Accumulator to Memory	1 0 1 0 0 0 1 w	addr-low	addr-high	
Register/Memory to Segment Register	1 0 0 0 1 1 1 0	mod 0 reg r/m		
Segment Register to Register/Memory	1 0 0 0 1 1 0 0	mod 0 reg r/m		

PUSH = Push:

Register/Memory	1 1 1 1 1 1 1 1	mod 1 1 0 r/m
Register	0 1 0 1 0 reg	
Segment Register	0 0 0 reg 1 1 0	

POP = Pop:

Register/Memory	1 0 0 0 1 1 1 1	mod 0 0 0 r/m
Register	0 1 0 1 1 reg	
Segment Register	0 0 0 reg 1 1 1	

❖ Data Transfer Instructions:

XCHG = Exchange:

Register/Memory with Register

1 0 0 0 0 1 1 w

mod reg r/m

Register with Accumulator

1 0 0 1 0 reg

IN = Input from:

Fixed Port

1 1 1 0 0 1 0 w

port

Variable Port

1 1 1 0 1 1 0 w

OUT = Output to:

Fixed Port

1 1 1 0 0 1 1 w

port

Variable Port

1 1 1 0 1 1 1 w

XLAT = Translate Byte to AL

1 1 0 1 0 1 1 1

LEA = Load EA to Register

1 0 0 0 1 1 0 1

mod reg r/m

LDS = Load Pointer to DS

1 1 0 0 0 1 0 1

mod reg r/m

LES = Load Pointer to ES

1 1 0 0 0 1 0 0

mod reg r/m

LAHF = Load AH with Flags

1 0 0 1 1 1 1 1

SAHF = Store AH into Flags

1 0 0 1 1 1 1 0

PUSHF = Push Flags

1 0 0 1 1 1 0 0

POPF = Pop Flags

1 0 0 1 1 1 0 1

❖ Data Transfer Instructions:

■ MOV Instruction:

- Instruction format: MOV Destination, Origin;
 $\text{Destination} \leftarrow \text{Origin}$
- Meaning: Transfers (copies) data from Origin to Destination
- Note: The two operands Destination and Origin must be compatible in size
- Examples:
 - MOV AL, 100; $\text{AL} \leftarrow 100$
 - MOV [BX], AH; $[\text{DS:BX}] \leftarrow \text{AH}$
 - MOV DS, AX; $\text{DS} \leftarrow \text{AX}$

❖ Data Transfer Instructions:

■ LODSB and LODSW Instructions:

- Instruction Format: LODSB; $AL \leftarrow [DS: SI]$ and $SI \leftarrow SI \pm 1$
- LODSW; $AX \leftarrow [DS: SI]$ and $SI \leftarrow SI \pm 2$
- Meaning: Loads the contents of the memory location whose address is in SI, belonging to the DS segment, into the AL/AX register and increments or decrements the contents of SI.
 - If $DF = 0$, it increments;
 - if $DF = 1$, it decrements.
- Examples:
 - `MOV SI, 1000; SI  $\leftarrow$  1000`
 - `MOV [DS:SI], 200; [DS:SI]  $\leftarrow$  200`
 - `CLD; DF  $\leftarrow$  0`
 - `LODSB; AL  $\leftarrow$  200; SI  $\leftarrow$  SI + 1`

❖ Data Transfer Instructions:

■ STOSB and STOSW instructions:

- Instruction format:
 - STOSB; $[ES:DI] \leftarrow AL$ $DI \leftarrow DI \pm 1$
 - STOSW; $[ES:DI] \leftarrow AX$ $DI \leftarrow DI \pm 2$
- Meaning: Stores the contents of register AL/AX in the memory location whose address is in DI within segment ES and increments or decrements the contents of DI.
 - If $DF = 0$, it increments;
 - if $DF = 1$, it decrements.
- Examples:
 - MOV DI, 1000; $DI \leftarrow 1000$
 - MOV AL, 200; $AL \leftarrow 200$
 - CLD; $DF \leftarrow 0$
 - STOSB; $[ES:DI] \leftarrow AL$; $DI \leftarrow DI + 1$

❖ Data Transfer Instructions:

■ MOVSB, MOVSW Instructions:

- Instruction format:
 - MOVSB; $[ES: DI] \leftarrow [DS: SI]$, $DI \leftarrow DI \pm 1$, $SI \leftarrow SI \pm 1$
 - MOVSW; $[ES: DI] \leftarrow [DS: SI]$, $DI \leftarrow DI \pm 2$, $SI \leftarrow SI \pm 2$
- Meaning: Transfers the contents of the memory location at address DS:SI to the memory location at address ES:DI and increments or decrements the contents of SI and DI.
 - If $DF = 0 \leftarrow$ increment,
 - $DF = 1 \leftarrow$ decrement.
- Examples:
 - MOV SI, 1000; $SI \leftarrow 1000$
 - MOV DI, 200; $DI \leftarrow 200$
 - CLD; $DF \leftarrow 0$
 - STOSB; $[ES:DI] \leftarrow [DS: SI]$

❖ Data Transfer Instructions:

■ IN instruction:

- Instruction format: `IN <register>, <input port address>`
- Meaning: Reads data from `<input port address>` and stores it in `<register>`.
- Numerical values can be used directly in the instruction if `<input port address>` is within the range 00-FFh;
- If `<input port address>` is greater than FFh, the port address needs to be stored in register DX.
- Examples:
 - `IN AL, 0F8H; AL ← (0F8h)`
 - `MOV DX, 02F8H;`
`IN AL, DX; AL ← (DX)`

❖ Data Transfer Instructions:

■ OUT Instruction:

- Instruction format: OUT <output port address>, <Root>
- Meaning: Saves data from Root to <output port address>.
- Numeric values can be used directly in the instruction if <output port address> is within the range 00-FFh;
- If <output port address> is greater than FFh, the port address needs to be stored in the DX register.
- Examples:
 - OUT 0F8H, AL; (0F8h) ← AL
 - MOV DX, 02F8H;
OUT DX, AL; (DX) ← AL

❖ Data Transfer Instructions:

- PUSH instruction – pushes data onto the stack
 - Format: PUSH <Root>
 - Meaning: Loads the Root onto the top of the stack; the Root must be a 2-byte operand.
 - Explanation
 - $SP \leftarrow SP + 2$; increments the stack pointer SP
 - $SP \leftarrow \text{Root}$; loads data onto the stack
 - Example: PUSH AX

❖ Data Transfer Instructions:

- POP instruction – retrieves data from the stack
 - Format: POP <Destination>
 - Meaning: Retrieves data from the top of the stack and stores it in Destination; Destination must be a 2-byte operand.
 - Explanation:
 - Destination $\leftarrow$ SP; retrieves data from the stack
 - SP $\leftarrow$ SP - 2; decrements the stack pointer SP
 - Example: POP BX

❖ Data Transfer Instructions:

- XCHG instruction – Swaps the values of two operands
 - Format: XCHG <Operand1>, <Operand2>
 - Meaning: Swaps the values of the two operands <Operand1> and <Operand2>
 - Example:
MOV BX, 100
MOV AX, 200
XCHG AX, BX; AX $\leftarrow$ 100, BX $\leftarrow$ 200

❖ Arithmetic Operations Instruction:

ARITHMETIC

ADD = Add:

Reg./Memory with Register to Either

7 6 5 4 3 2 1 0

7 6 5 4 3 2 1 0

7 6 5 4 3 2 1 0

7 6 5 4 3 2 1 0

0 0 0 0 0 0 d w

mod reg r/m

Immediate to Register/Memory

1 0 0 0 0 0 s w

mod 0 0 0 r/m

data

data if s: w = 01

Immediate to Accumulator

0 0 0 0 0 1 0 w

data

data if w = 1

ADC = Add with Carry:

Reg./Memory with Register to Either

0 0 0 1 0 0 d w

mod reg r/m

Immediate to Register/Memory

1 0 0 0 0 0 s w

mod 0 1 0 r/m

data

data if s: w = 01

Immediate to Accumulator

0 0 0 1 0 1 0 w

data

data if w = 1

INC = Increment:

Register/Memory

1 1 1 1 1 1 1 w

mod 0 0 0 r/m

Register

0 1 0 0 0 reg

AAA = ASCII Adjust for Add

0 0 1 1 0 1 1 1

BAA = Decimal Adjust for Add

0 0 1 0 0 1 1 1

❖ Arithmetic Operations Instruction:

SUB = Subtract:

Reg./Memory and Register to Either

0 0 1 0 1 0 d w	mod reg r/m
-----------------	-------------

Immediate from Register/Memory

1 0 0 0 0 0 s w	mod 1 0 1 r/m	data	data if s w = 01
-----------------	---------------	------	------------------

Immediate from Accumulator

0 0 1 0 1 1 0 w	data	data if w = 1
-----------------	------	---------------

SSB = Subtract with Borrow

Reg./Memory and Register to Either

0 0 0 1 1 0 d w	mod reg r/m
-----------------	-------------

Immediate from Register/Memory

1 0 0 0 0 0 s w	mod 0 1 1 r/m	data	data if s w = 01
-----------------	---------------	------	------------------

Immediate from Accumulator

0 0 0 1 1 1 w	data	data if w = 1
---------------	------	---------------

DEC = Decrement:

Register/memory

1 1 1 1 1 1 1 w	mod 0 0 1 r/m
-----------------	---------------

Register

0 1 0 0 1 reg

NEG = Change sign

1 1 1 1 0 1 1 w	mod 0 1 1 r/m
-----------------	---------------

❖ Arithmetic Operations Instruction:

CMP = Compare:

Register/Memory and Register

001110dw	mod reg r/m
----------	-------------

Immediate with Register/Memory

100000sw	mod 111 r/m	data	data if s w = 01
----------	-------------	------	------------------

Immediate with Accumulator

0011110w	data	data if w = 1
----------	------	---------------

AAS = ASCII Adjust for Subtract

00111111

DAS = Decimal Adjust for Subtract

00101111

MUL = Multiply (Unsigned)

1111011w	mod 100 r/m
----------	-------------

IMUL = Integer Multiply (Signed)

1111011w	mod 101 r/m
----------	-------------

AAM = ASCII Adjust for Multiply

11010100	00001010
----------	----------

DIV = Divide (Unsigned)

1111011w	mod 110 r/m
----------	-------------

IDIV = Integer Divide (Signed)

1111011w	mod 111 r/m
----------	-------------

AAD = ASCII Adjust for Divide

11010101	00001010
----------	----------

CBW = Convert Byte to Word

10011000

CWD = Convert Word to Double Word

10011001

❖ Arithmetic Operations Instruction:

■ The ADD instruction - adding integers:

- Instruction format: ADD <Destination>, <Root>;

$$\text{Destination} \leftarrow \text{Destination} + \text{Root}$$

- Meaning: Adds Root to Destination, the result is stored in Destination

- The ADD instruction affects the flags: C, Z, S, P, O, A

- Examples:

–ADD AX, BX; $AX \leftarrow AX + BX$

–ADD AL, 10; $AL \leftarrow AL + 10$

–ADD [BX], AL; $[DS:BX] \leftarrow [DS:BX] + AL$ Today we begin to learn about the group of instructions that perform arithmetic operations in the 8086 microprocessor.

❖ Arithmetic Operations Instruction:

- The SUB command - subtracts integers:
 - Command format: SUB <Destination>, <Root>;
$$\text{Destination} \leftarrow \text{Destination} - \text{Root}$$
 - Meaning: Subtracts the Destination from the Root, and stores the result in Destination
 - The SUB command affects the flags: C, Z, S, P, O, A
 - Examples:
 - SUB AX, BX; $AX \leftarrow AX - BX$
 - SUB AL, 10; $AL \leftarrow AL - 10$
 - SUB [BX], AL; $[DS:BX] \leftarrow [DS:BX] - AL$

❖ Arithmetic Operations Instruction:

- The MUL instruction - multiplying integers:
 - Instruction format: MUL <Root>;
The root must be a register or memory address
 - Meaning:
 - If the root is 8 bits: $AX \leftarrow AL * \text{Root}$
 - If the root is 16 bits: $AX \leftarrow AX * \text{Root}$
 - The MUL instruction affects the flags: Z, S, P
 - Example: calculate $10 * 30$
MOV AL, 10; $AL \leftarrow 10$
MOV BL, 30; $BL \leftarrow 30$
MUL BL; $AX \leftarrow AL * BL$

❖ Arithmetic Operations Instruction:

- Increment instruction (INC):
 - Format: INC <Destination>; $\text{Destination} \leftarrow \text{Destination} + 1$
- Decrement instruction (DEC):
 - Format: DEC <Destination>; $\text{Destination} \leftarrow \text{Destination} - 1$
- Comparison instruction (CMP):
 - Format: CMP <Destination>, <Origin>
 - Meaning: Calculates $\text{Destination} - \text{Origin}$; the result is only used to update the status flag bits, not stored in Destination:

Case	C	Z	S
Destination > Origin	0	0	0
Destination = Origin	0	1	0
Destination < Origin	1	0	1

❖ Arithmetic Operations Instruction:

- NEG instruction – reverses the sign of the operand
 - Format: NEG <Destination>
 - Meaning: Reverses the sign of the value stored in Destination
 - Example:

MOV AX, 1000; $AX \leftarrow 1000$

NEG AX; $AX \leftarrow -(AX) = -1000$

❖ Logical Instructions:

LOGIC

NOT = Invert

SHL/SAL = Shift Logical/Arithmetic Left

SHR = Shift Logical Right

SAR = Shift Arithmetic Right

ROL = Rotate Left

ROR = Rotate Right

RCL = Rotate Through Carry Flag Left

RCR = Rotate Through Carry Right

7 6 5 4 3 2 1 0	7 6 5 4 3 2 1 0
1 1 1 1 0 1 1 w	mod 0 1 0 r/m
1 1 0 1 0 0 v w	mod 1 0 0 r/m
1 1 0 1 0 0 v w	mod 1 0 1 r/m
1 1 0 1 0 0 v w	mod 1 1 1 r/m
1 1 0 1 0 0 v w	mod 0 0 0 r/m
1 1 0 1 0 0 v w	mod 0 0 1 r/m
1 1 0 1 0 0 v w	mod 0 1 0 r/m
1 1 0 1 0 0 v w	mod 0 1 1 r/m

❖ Logical Instructions:

AND = And:

Reg./Memory and Register to Either

001000dw	mod reg r/m
----------	-------------

Immediate to Register/Memory

1000000w	mod 100 r/m	data	data if w = 1
----------	-------------	------	---------------

Immediate to Accumulator

0010010w	data	data if w = 1
----------	------	---------------

TEST = And Function to Flags, No Result:

Register/Memory and Register

1000010w	mod reg r/m
----------	-------------

Immediate Data and Register/Memory

1111011w	mod 000 r/m	data	data if w = 1
----------	-------------	------	---------------

Immediate Data and Accumulator

1010100w	data	data if w = 1
----------	------	---------------

OR = Or:

Reg./Memory and Register to Either

000010dw	mod reg r/m
----------	-------------

Immediate to Register/Memory

1000000w	mod 001 r/m	data	data if w = 1
----------	-------------	------	---------------

Immediate to Accumulator

0000110w	data	data if w = 1
----------	------	---------------

XOR = Exclusive or:

Reg./Memory and Register to Either

001100dw	mod reg r/m
----------	-------------

Immediate to Register/Memory

1000000w	mod 110 r/m	data	data if w = 1
----------	-------------	------	---------------

Immediate to Accumulator

0011010w	data	data if w = 1
----------	------	---------------

❖ Logical Instructions:

■ NOT Instruction:

- Instruction format: NOT <Destination>
- Meaning: Inverts the bits of the Destination operand
- NOT instruction affects the flags: Z, S, P
- Examples:

MOV AL, 80H; 80H = 1000 0000B

NOT AL; 7FH = 0111 1111B

❖ Logical Instructions:

■ AND Instruction:

- Instruction format: AND <Destination>, <Source>
- Meaning: Multiplies the bit pairs of the two operands Destination and Source, and transfers the result to Destination.
- The AND instruction affects the flags: Z, S, P.
- Examples: AND can be used to clear one or more bits:
 - Clear the 3rd bit of the AL register (0-7):
`AND AL, F7H; F7H = 1111 0111B`
 - Clear the 4 high bits of the AL register (0-7):
`AND AL, 0FH; 0FH = 0000 1111B`

❖ Logical Instructions:

■ OR Instruction:

- Instruction format: OR <Destination>, <Source>
- Meaning: Adds the bit pairs of the two operands Destination and Source; the result is passed to Destination.
- The OR instruction affects the flags: Z, S, P.
- Examples: OR can be used to set one or more bits:
 - Set the 3rd bit of the AL register (0-7):
OR AL, 08H; 08H = 0000 1000B
 - Set the 7th bit of the AL register (0-7):
OR AL, 80H; 80H = 1000 0000B

❖ Logical Instructions:

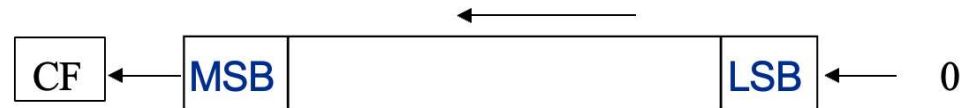
■ XOR Instruction:

- Instruction format: XOR <Destination>, <Source>
- Meaning: Adds and inverts the bit pairs of the two operands Destination and Source; the result is transferred to Destination.
- The XOR instruction affects the flags: Z, S, P.
- Example: Using XOR to clear the contents of a register/memory cell:
 - Clear register AL (0-7):
XOR AL, AL;
 - Clear register BX:
XOR BX, BX ; AL $\leftarrow$ 0, BX $\leftarrow$ 0

❖ Shift and Rotate Instructions: These include the following instructions:

- Shift instructions are often used to replace multiplication (left shift) and division (right shift).
 - Shift Left: SHL
 - Shift Right: SHR
- Rotate Instructions
 - Rotate Left: ROL
 - Rotate Right: ROR
- Shift and rotate instructions can also be used when bit-by-bit processing is required.

❖ Shift and Rotate Instructions:



■ Left Shift Instruction: SHL

- Instruction Format: SHL <Destination>, 1
 SHL <Destination>, CL
- Meaning: Shifts left by one bit or a number of bits in CL if shifted by more than 1 bit.
 - MSB (Most Significant Bit) is moved to the carry flag CF.
 - 0 is filled into LSB (Least Significant Bit).
 - Bits between MSB and LSB are shifted left by 1 bit.
- Examples:
 - MOV AL, 08H; 0000 1000B
 SHL AL, 1; 0001 0000B
 - MOV CL, 2;
 SHL AL, CL; 0100 0000B

❖ Shift and Rotate Instructions:

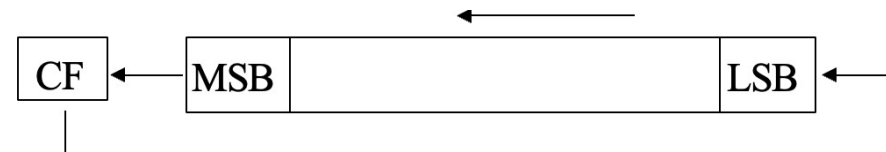
■ Right Shift Instruction: SHR



- Instruction Format: SHR <Destination>, 1
 SHR <Destination>, CL
- Meaning: Shift right by one bit or number of bits in CL if shifted by more than 1 bit
 - LSB (Least Significant Bit) moves to carry flag CF
 - 0 is filled in MSB (Most Significant Bit)
 - Bits between MSB and LSB are shifted right by 1 bit
- Examples:
 - MOV AL, 08H; 1000 0000B (128)
 SHR AL, 1; 0100 0000B (64)
 - MOV CL, 2;
 SHR AL, CL; 0001 0000B (16)

❖ Shift and Rotate Instructions:

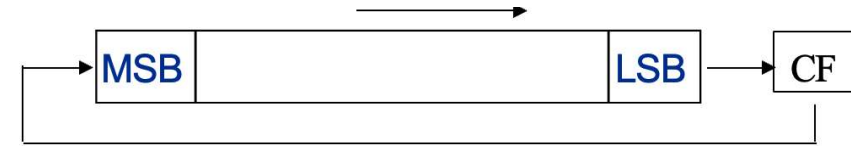
■ Left Rotation Instruction: ROL



- Instruction Format: ROL <Destination>, 1
 ROL <Destination>, CL
- Meaning: Rotates left by one bit or a number of bits in CL if turned by more than 1 bit
 - MSB (Most Significant Bit) is moved to the carry flag CF
 - MSB is moved to LSB (Least Significant Bit)
 - Bits between MSB and LSB are shifted left by 1 bit
- Examples:
 - MOV AL, 88H; 1000 1000B
 ROL AL, 1; 0001 0001B
 - MOV CL, 2;
 ROL AL, CL; 0100 0100B

❖ Shift and Rotate Instructions:

■ Right Rotation Instruction: ROR



- Instruction format: `ROR <Destination>, 1`
`ROR <Destination>, CL`
- Meaning: Rotates right one bit or more bits in CL if turned more than 1 bit
 - LSB (Least Significant Bit) moves to carry flag CF
 - LSB is moved to MSB (Most Significant Bit)
 - Bits between MSB and LSB are shifted right by 1 bit
- Examples:
- `MOV AL, 88H; 1000 1000B`
`ROR AL, 1; 0100 0100B`
- `MOV CL, 2;`
`ROR AL, CL; 0001 0001B`

❖ Program flow control instructions

JMP = Unconditional Jump:

Direct within Segment

7 6 5 4 3 2 1 0

1 1 1 0 1 0 0 1

7 6 5 4 3 2 1 0

disp-low

7 6 5 4 3 2 1 0

disp-high

Direct within Segment-Short

1 1 1 0 1 0 1 1

disp

Indirect within Segment

1 1 1 1 1 1 1 1

mod 1 0 0 r/m

Direct Intersegment

1 1 1 0 1 0 1 0

offset-low

offset-high

seg-low

seg-high

Indirect Intersegment

1 1 1 1 1 1 1 1

mod 1 0 1 r/m

RET = Return from CALL:

Within Segment

1 1 0 0 0 0 1 1

Within Seg Adding Immed to SP

1 1 0 0 0 0 1 0

data-low

data-high

Intersegment

1 1 0 0 1 0 1 1

Intersegment Adding Immediate to SP

1 1 0 0 1 0 1 0

data-low

data-high

❖ Program flow control instructions

JE/JZ = Jump on Equal/Zero	0 1 1 1 0 1 0 0	disp
JL/JNGE = Jump on Less/Not Greater or Equal	0 1 1 1 1 1 0 0	disp
JLE/JNG = Jump on Less or Equal/Not Greater	0 1 1 1 1 1 1 0	disp
JB/JNAE = Jump on Below/Not Above or Equal	0 1 1 1 0 0 1 0	disp
JBE/JNA = Jump on Below or Equal/Not Above	0 1 1 1 0 1 1 0	disp
JP/JPE = Jump on Parity/Parity Even	0 1 1 1 1 0 1 0	disp
JO = Jump on Overflow	0 1 1 1 0 0 0 0	disp
JS = Jump on Sign	0 1 1 1 1 0 0 0	disp
JNE/JNZ = Jump on Not Equal/Not Zero	0 1 1 1 0 1 0 1	disp
JNL/JGE = Jump on Not Less/Greater or Equal	0 1 1 1 1 1 0 1	disp
JNLE/JG = Jump on Not Less or Equal/Greater	0 1 1 1 1 1 1 1	disp
JNB/JAE = Jump on Not Below/Above or Equal	0 1 1 1 0 0 1 1	disp
JNBE/JA = Jump on Not Below or Equal/Above	0 1 1 1 0 1 1 1	disp
JNP/JPO = Jump on Not Par/Par Odd	0 1 1 1 1 0 1 1	disp

❖ Program flow control instructions

JNO = Jump on Not Overflow

0 1 1 1 0 0 0 1

disp

JNS = Jump on Not Sign

0 1 1 1 1 0 0 1

disp

LOOP = Loop CX Times

1 1 1 0 0 0 1 0

disp

LOOPZ/LOOPE = Loop While Zero/Equal

1 1 1 0 0 0 0 1

disp

LOOPNZ/LOOPNE = Loop While Not
Zero/Equal

1 1 1 0 0 0 0 0

disp

JCXZ = Jump on CX Zero

1 1 1 0 0 0 1 1

disp

INT = Interrupt

Type Specified

1 1 0 0 1 1 0 1

type

Type 3

1 1 0 0 1 1 0 0

INTO = Interrupt on Overflow

1 1 0 0 1 1 1 0

IRET = Interrupt Return

1 1 0 0 1 1 1 1

❖ Program flow control instructions

PROCESSOR CONTROL

CLC = Clear Carry

7 6 5 4 3 2 1 0

1 1 1 1 1 0 0 0

CMC = Complement Carry

1 1 1 1 0 1 0 1

STC = Set Carry

1 1 1 1 1 0 0 1

CLD = Clear Direction

1 1 1 1 1 1 0 0

STD = Set Direction

1 1 1 1 1 1 0 1

CLI = Clear Interrupt

1 1 1 1 1 0 1 0

STI = Set Interrupt

1 1 1 1 1 0 1 1

HLT = Halt

1 1 1 1 0 1 0 0

WAIT = Wait

1 0 0 1 1 0 1 1

ESC = Escape (to External Device)

1 1 0 1 1 x x x

mod x x x r/m

LOCK = Bus Lock Prefix

1 1 1 1 0 0 0 0

7 6 5 4 3 2 1 0

❖ Program flow control instructions

CONTROL TRANSFER

CALL = Call:

Direct within Segment

1 1 1 0 1 0 0 0	disp-low	disp-high
-----------------	----------	-----------

Indirect within Segment

1 1 1 1 1 1 1 1	mod 0 1 0 r/m	
-----------------	---------------	--

Direct Intersegment

1 0 0 1 1 0 1 0	offset-low	offset-high
-----------------	------------	-------------

	seg-low	seg-high
--	---------	----------

Indirect Intersegment

1 1 1 1 1 1 1 1	mod 0 1 1 r/m	
-----------------	---------------	--

❖ Program flow control instructions: are instructions that change the order of program execution

These include:

- Unconditional jump instruction JMP
- Conditional jump instructions JE, JZ, JNE, JNZ, JL, JLE, JG, JGE, ...
- Loop instructions LOOP, LOOPE, LOOPZ
- Call to execute a subroutine CALL
- Return instruction from a subroutine RET

❖ Program flow control instructions:

■ Unconditional Jump Instruction JMP

- Instruction format: `JMP <label>`
- Meaning: Jumps to execute the instruction immediately following `<label>`.
 - `<label>` is a name placed before an instruction, separated by colons (:).
- JMP jump ranges can be short (-128 : +127), near (-32768 : +32767), and far (using the full address CS:IP).
- Example:

START: ADD AX, BX

SUB BX, 1

.....

JMP START; jumps to execute the instruction after the label

START

❖ Program flow control instructions:

- Conditional jump instructions JE, JZ, JNE, JNZ, JL, JG
- Instruction format:
 - JE <label> : jump if equal or result is 0
 - JZ <label> : jump if equal or result is 0
 - JNE <label> : jump if not equal or result is not 0
 - JNZ <label> : jump if not equal or result is not 0
 - JL <label> : jump if less than
 - JLE <label> : jump if less than or equal to
 - JG <label> : jump if greater than
 - JGE <label> : jump if greater than or equal to
- The jump range of conditional jump instructions is short (-128 : +127).

❖ Program flow control instructions:

- Example: Write a program to calculate the sum of numbers from 1-20.

MOV AX, 0; AX contains the sum.

MOV BX, 20; set the value for the counter variable BX.

START: ADD AX, BX; add cumulatively.

SUB BX, 1; decrease the counter variable.

JZ STOP; stop if BX = 0.

JMP START; return to the next loop.

STOP:

❖ Program flow control instructions:

■ LOOP instruction

- Instruction format: LOOP <label>
- Meaning: Moves to execute the instruction immediately following <label> if the value in register CX is not 0. Automatically decrements the value of CX by 1 unit upon execution.
- Example: Write a program to calculate the sum of numbers from 1-20:

MOV AX, 0; AX contains the sum

MOV CX, 20; set the value for the counter variable CX

START: ADD AX, CX; accumulate

LOOP START; check CX, if CX=0 → stop;

if CX is not 0: CX ← CX-1 and return;
start a new loop from START

❖ Program flow control instructions:

- CALL and RET Instructions
- Instruction Format:
 - CALL <subroutine name>: calls and executes the subroutine
 - RET: returns from the subroutine; usually placed at the end of the subroutine

■ Example:

CALL GIAITHUA; calls the subroutine GIAITHUA

..... ; part of the main program code

GIAITHUA PROC; begins the code of the subroutine

.....

RET; returns to the calling program

GIAITHUA ENDP; ends the code of the subroutine

- ❖ Bit manipulation instructions include a group of instructions that manipulate several bits (D, C, I) of the FR flag register;
 - Flag setting instructions (set flag bit to 1)
 - STD: set D direction flag
 - STC: set C carry flag
 - STI: set I interrupt flag
 - Flag clearing instructions (set flag bit to 0)
 - CLD: clear D direction flag
 - CLC: clear C carry flag
 - CLI: clear I interrupt flag

❖ Other instructions: NOP and Halt

- NOP (No Operation) instruction:
 - NOP does not perform a specific task, it only consumes time equal to one instruction cycle
- HLT (Halt) instruction:
 - HLT stops program execution
- INT instruction – Invoking interrupt service routine
 - Format: INT <Interrupt Number>
 - Meaning: Calls the interrupt service routine corresponding to <Interrupt Number>
 - Examples:

MOV AH, 4Ch; Loads function 4Ch

INT 21h; Calls DOS interrupt number 21h

❖ String Instructions

STRING MANIPULATION

REP = Repeat

1 1 1 1 0 0 1 z

MOVS = Move Byte/Word

1 0 1 0 0 1 0 w

CMPS = Compare Byte/Word

1 0 1 0 0 1 1 w

SCAS = Scan Byte/Word

1 0 1 0 1 1 1 w

LODS = Load Byte/Wd to AL/AX

1 0 1 0 1 1 0 w

STOS = Stor Byte/Wd from AL/A

1 0 1 0 1 0 1 w

❖ The REP instruction repeats the execution of the MOVSB, MOVSW, LODSB, LODSW, STOSB, STOSW instructions a certain number of times – the number of repetitions is stored in the CX register.

- Format: REP <Instruction to repeat>
- Meaning: Repeats the execution of another instruction CX times
- Examples:

MOV SI, 1000; Sets the source address

MOV DI, 2000; Sets the destination address

MOV CX, 10; Sets the number of repetitions for REP

REP MOVSB; Executes MOVSB 10 times: transfers the contents of 10 memory locations starting from DS:SI to 10 memory locations starting from ES:DI

- ❖ Assembly language is a low-level programming language, only slightly higher than machine language;
 - Assembly language is closely tied to microprocessors (processor-specific).
 - Instructions used in assembly language are microprocessor instructions.
 - An assembly program written for one microprocessor may not work on another.
 - Assembly programs, when compiled into machine code, are compact and occupy little memory space.
 - Assembly language is commonly used to write:
 - Device drivers
 - Program modules for microcontrollers
 - Some modules in the operating system kernel (requiring compact size and high speed)

❖ Assembly Program Syntax

- In an assembly program, each instruction is placed on a separate line – the instruction line;
- Instructions have two forms:
 - Real instructions: These are mnemonic instructions of the microcontroller
 - Examples: MOV, SUB, ADD,...
 - When compiled, the mnemonic instructions are translated into machine code
 - Pseudo-instructions: These are instructions for the compiler
 - Examples: MAIN PROC, .DATA, END MAIN,...
 - When compiled, pseudo-instructions are not translated into machine code; the code only serves to guide the compiler.
- Case sensitivity is not a factor in assembly program instructions during compilation.

❖ Assembly language instruction structure:

[Name] [Instruction Code] [Operators] [Comment]

START:	MOV	AH, 100	; Chuyển 100 vào thanh ghi AH
↑	↑	↑	↑
Tên	Mã lệnh	Toán hạng	Chú giải

■ Instruction line fields:

• Name:

- Is a label, variable name, constant, or procedure. A colon (:) follows the label.
- Names will be assigned memory addresses by the compiler.
- Names can only consist of letters, numbers, underscores, and must begin with a letter.

❖ Assembly language instruction structure:

- Instruction line fields:
 - Instruction Code: can include both real and pseudo-instructions.
 - Operands:
 - The number of operands depends on the specific instruction.
 - There can be 0, 1, or 2 operands.
 - Comments:
 - Comments for the command line.
 - Begin with a semicolon (;)

❖ Data for assembly language programs

■ Numerical data:

- Decimal: 0-9
- Hexadecimal: 0-9, A-F
 - If starting with a letter (A-F), add a 0 at the beginning
 - Add the symbol H (Hexadecimal) at the end
 - Examples: 80H, 0F9H
- Binary: 0-1
 - Add the symbol B (Binary) at the end
 - Examples: 0111B, 1000B
- Character data:
 - Enclosed in single or double quotes
 - Can be used as characters or ASCII code
 - » 'A' = 65, 'a' = 97

❖ Data for assembly language programs

- Constants:
 - Quantities whose values do not change
 - Two types of constants:
 - Value constants: e.g., 100, 'A'
 - Named constants: e.g., MAX_VALUE
 - Definition of a named constant: `<Constant Name> EQU <Value>`
 - Example:
 - MAX EQU 100
 - ENTER EQU 13
 - ESC EQU 27

❖ Data for assembly language programs

- Variables:
 - Quantities whose values can change
 - Types of variables:
 - Simple variables
 - Array variables
 - String variables
 - When compiling, variables are converted into memory addresses

❖ Data for assembly language programs

■ Variables:

• Definition of simple variables:

- Variable name DB Initial value: Defines a byte variable
- Variable name DW Initial value: Defines a word variable
- Variable name DD Initial value: Defines a double word variable

• Examples:

- X DB 10; Declares variable X and initializes it to 10
- Y DW ?; Declares variable Y and does not initialize it
- Z DD 1000; Declares variable X and initializes it to 1000

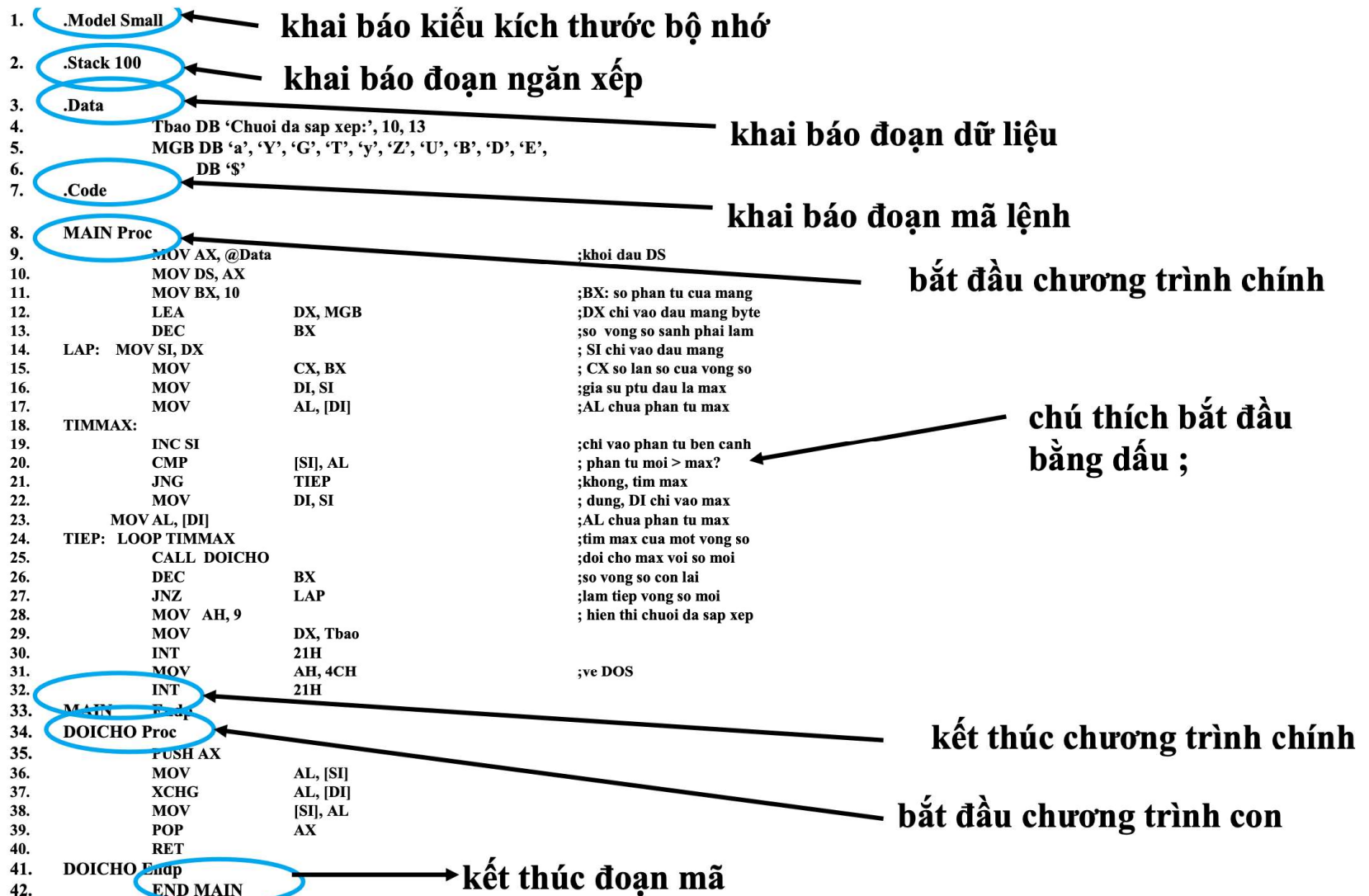
❖ Data for assembly language programs

- Array variable definition:
 - Array name DB Initialized value list
 - Array name DB Number of elements Dup(Initialized value)
 - Array name DB Number of elements Dup(?)
 - Similar definition for DW and DD types
 - Examples:
 - X DB 10, 2, 5, 6, 1; Declares array X with 5 elements initialized
 - Y DB 5 DUP(0); Declares array Y with 5 elements initialized 0
 - Z DB 5 DUP(?); Declares array Z with 5 elements not initialized

❖ Data for assembly language programs

- Defining string variables: can be defined as a string or an array of characters.
 - Examples:
 - str1 DB 'string'
 - str2 DB 73H, 74H, 72H, 69H, 6EH, 67H
 - str3 DB 73H, 74H, 'r', 'i', 69H, 6EH, 67H

❖ Assembly Language Framework



❖ Assembly Language Framework

- Declaring memory usage scale:

.Model <Memory size type>

- Memory size types:

- Tiny: code and data are contained in a single segment
- Small: code is contained in a single segment, data is contained in a single segment
- Medium: code is not contained in a single segment, data is contained in a single segment
- Compact: code is contained in a single segment, data is not contained in a single segment
- Large: code is not contained in a single segment, data is not contained in a single segment, no arrays larger than 64K
- Huge: code is not contained in a single segment, data is not contained in a single segment, arrays larger than 64K

❖ Assembly Language Framework

■ Declaring a stack segment:

`.Stack <Stack size>`

- Example: `.Stack 100H`; declares stack size `100H = 256 bytes`

■ Declaring a data segment:

`.Data`; Defines variables and constants

; All variables and constants must be declared in the data segment

- Example:

`.Data MSG DB 'Hello!$'`

`ENTER DB 13`

`MAX DW 1000`

❖ Assembly Language Framework

- Declaring the code segment:

.Code

Includes the program commands:

.Code

Jmp Start

; data declaration

Start:

MOV AX,@Data

MOV DS, AX

; commands of the main program

MOV AH, 4CH

INT 21H

End Start ; end of the main program

; subroutines – if any

❖ Assembly Language Framework

```

1  .Model Small
2  .Stack 100H
3  .Data
4      ; Khai bao cac bien va hang
5  .Code
6      jmp Start
7  Start:
8      ; Khoi tao cho thanh ghi DS
9      MOV AX, @Data; nap dia chi doan du lieu vao AX
10     MOV DS, AX; nap dia chi doan du lieu vao DS
11
12     ; Cac lenh cua chuong trinh chinh
13
14     ; Ket thuc tro ve chuong trinh goi
15     ; Dung ham 4CH cua ngat 21H
16     MOV AH, 4CH
17     INT 21H
18 End Start
19     ; Cac chuong trinh con neu co

```

❖ Assembly Language Framework

```

1      .Model Small
2      .Stack 100H
3      .Data
4          ; Khai bao cac bien va hang
5      CRLF DB 13,10,'$'; Ky tu xuong dong
6      MSG DB 'Hello_World!$'
7      .Code
8      MAIN Proc
9          ; Khoi tao cho thanh ghi DS
10         MOV AX, @Data; nap dia chi doan du lieu vao AX
11         MOV DS, AX; nap dia chi doan du lieu vao DS
12         ; In gia tri xuong dong
13         MOV AH, 9
14         LEA DX, CRLF; Nap dia chi CRLF vao DX
15         INT 21H
16         ; Hien loi chao dung ham 9 cua ngat 21H
17         MOV AH, 9
18         LEA DX, MSG; Nap dia chi cua thong diep vao DX
19         INT 21H; hien thong diep
20         ; Ket thuc tro ve chuong trinh goi
21         ; dung ham 4CH cua ngat 21H
22         MOV AH, 4CH
23         INT 21H
24     MAIN Endp

```


❖ Assembly Language Framework

 Terminal

```
Hello World!
```

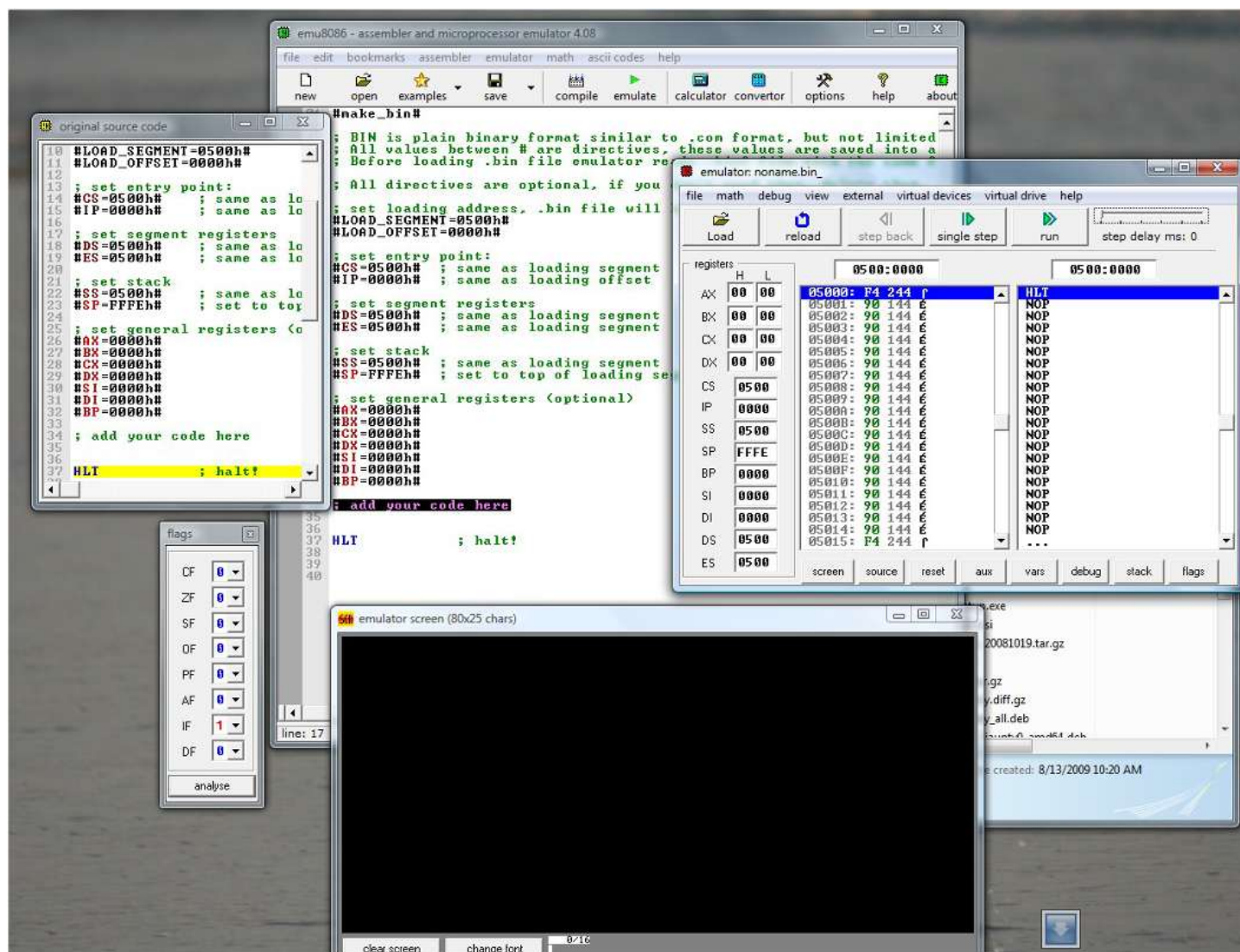
❖ Some services of interrupt 21H:

- Function 1 of interrupt INT 21H: Reads 1 character from keyboard
 - Input: $AH = 1$
 - Output: $AL = \text{ASCII code of the character to be displayed.}$
 $AL = 0$ when the input character is a function key.
- Function 2 of interrupt INT 21H: Displays 1 character on screen
 - Input: $AH = 2$
 $DL = \text{ASCII code of the character to be displayed.}$
 - Output: None.

❖ Some services of interrupt 21H:

- Function 4CH of interrupt INT 21H: Terminates EXE program
 - Input: AH = 4CH
 - Output: None.
- Function 9 of interrupt INT 21H: Displays a string of characters with a \$ at the end on screen
 - Input: AH = 9
DX = offset address of the string of characters to be displayed.
 - Output: None.

❖ Characteristics



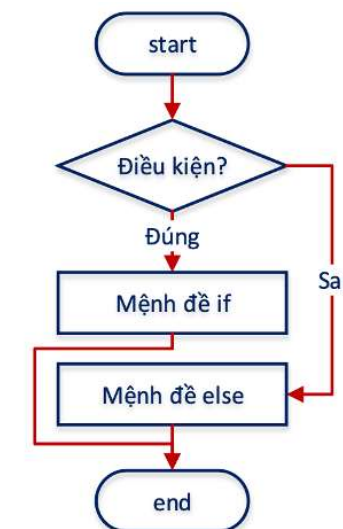
❖ Some control structures and examples

■ Selection structures

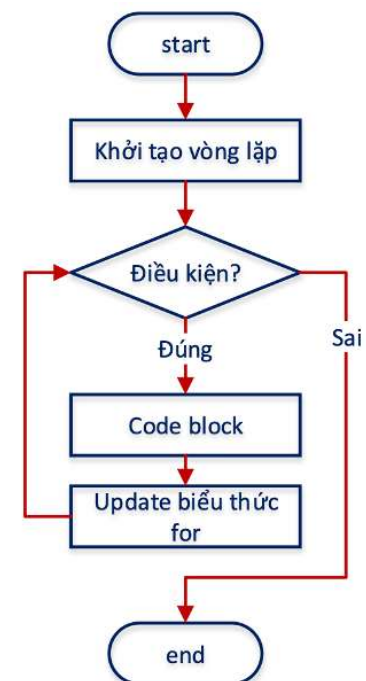
- IF ... THEN branching
- IF ... THEN ... ELSE branching
- Multiple branches

■ Loop structures

- For loop
- Repeat ... until loop



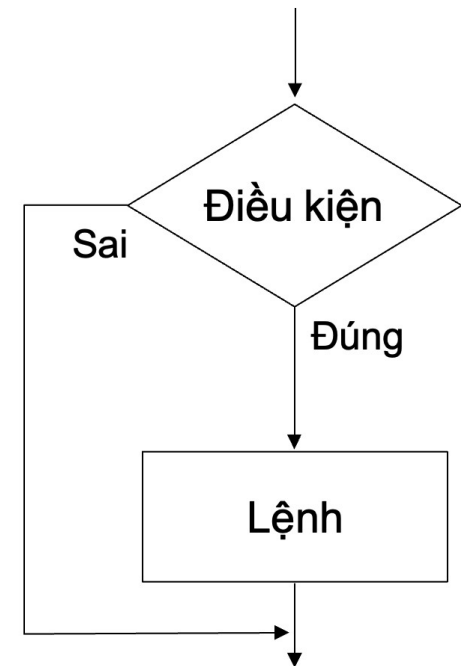
Hình 1: Cấu trúc lệnh if/else



Hình 2: cấu trúc lệnh for

❖ Some control structures and examples

- IF ... THEN branching:
 - IF condition THEN operation

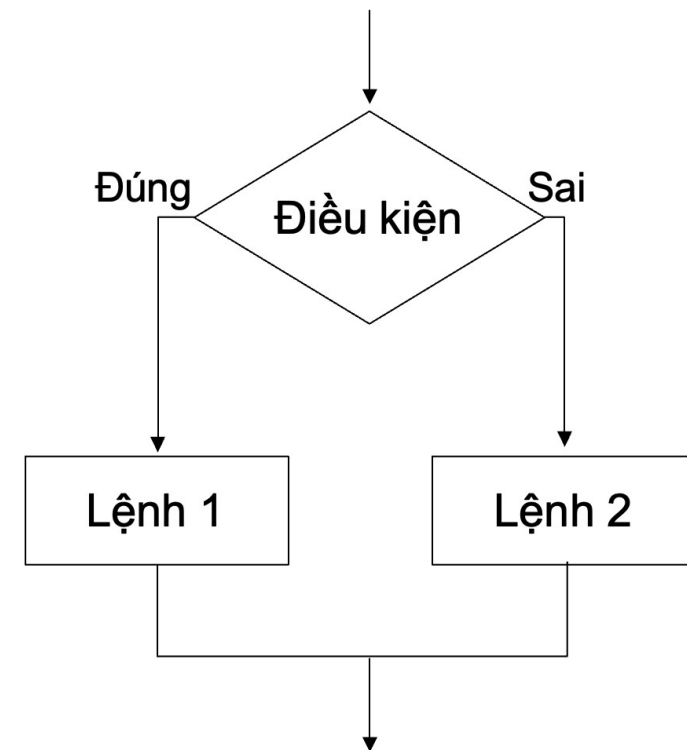


► Assign absolute value BX to AX

```
1  CMP AX,0
2  JNL GAN
3  NEG AX
4  GAN:
5  MOV BX, AX
```

❖ Some control structures and examples

■ IF ... THEN ... ELSE branching:



► Assign the sign bit of AX to CL:

```
1  CMP AX, 0 ; AX > 0 ?
2  JNS DG ; Dung
3  MOV CL, 1 ; Khong, CL <-- 1
4  JMP RA ; Nhay qua nhanh kia
5  DG: MOV CL, 0 ; CL <-- 0
6  RA:
```

❖ Some control structures and examples

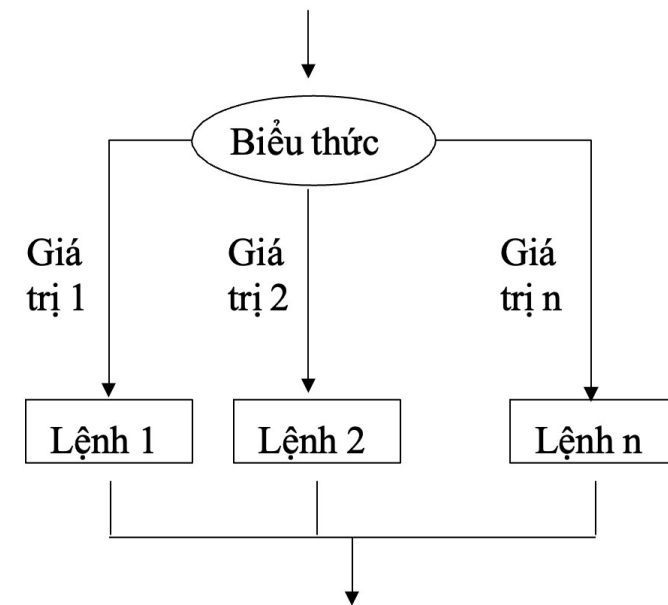
■ Multiple branches:

► Assign a value to CX according to the rule:

If $AX < 0$ then $CX = -1$

If $AX = 0$ then $CX = 0$

If $AX > 0$ then $CX = 1$

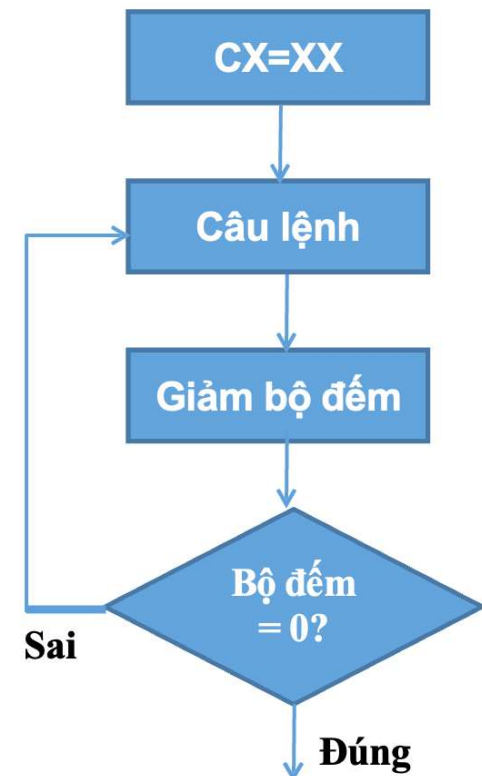


```
1  CMP AX, 0
2  JL  AM
3  JE  KHONG
4  JG  DUONG
5  AM: MOV CX, -1
6      JMP RA
7  DUONG: MOV CX, 1
8      JMP RA
9  KHONG: MOV CX, 0
10 RA:
```


❖ Some control structures and examples

■ For loop:

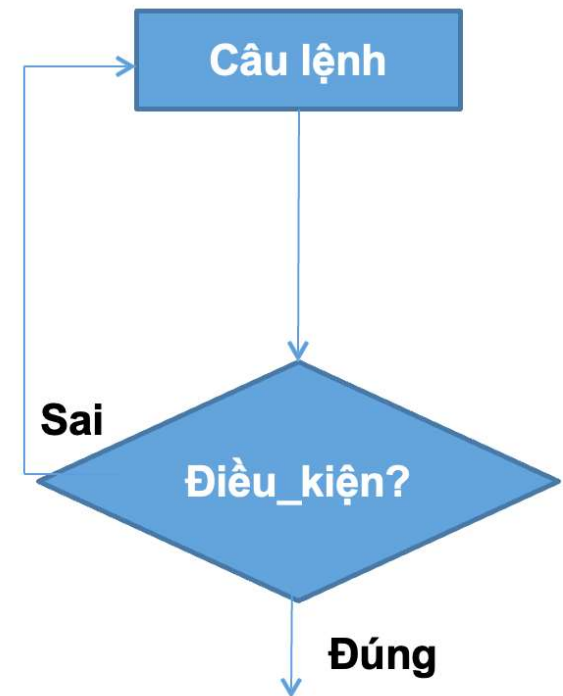
- Use the LOOP statement
- Number of iterations CX



```
1  MOV CX,10
2  MOV AH,2
3  MOV DL,'9'
4  Hien: INT 21H
5  LOOP Hien
```

❖ Some control structures and examples

- Repeat ... until loop:
 - 1. . . .
 - 2. Continue: . . .
 - 3.
 - 4. `CMP X,Y; condition`
 - Return to Continue if `condition=false`;



❖ Exercise 1 - Arithmetic Instructions: Write a program that takes a value of x as input and calculates the value of the expression f below. The result should be stored in register AH and printed to the screen (assuming x is small enough so that the result of the expression does not exceed 8 bits; for simplicity, experiment with $x = 0$ and $x = 1$).

$$f = ax^3 - bx^2 - cx + d$$

với $a = 4, b = 3, c = 2, d = 1$

Gợi ý: (theo phương pháp Horner's Method, *sinh viên có thể làm theo cách của riêng mình*)

- Khởi tạo giá trị cho a, b, c, d, x bằng lệnh số học: add/addi, orx
- Nhân a với x rồi lưu kết quả vào thanh ghi tạm. $t = a * x$
- Thực hiện phép số tính giữa thanh ghi tạm với b . $t = t - b$ // $t = ax - b$
- Nhân thanh ghi tạm với x . $t = t * x$ // $t = (ax - b)x$
- Thực hiện phép số tính giữa thanh ghi tạm với c . $t = t - c$ // $t = ax^2 - bx - c$
- Nhân thanh ghi tạm với x . $t = t * x$ // $t = (ax^2 - bx - c)x$
- Thực hiện phép số tính giữa thanh ghi tạm với d . $t = t + d$ // $t = ax^3 - bx^2 - cx + d$

❖ Exercise 2 - Arithmetic Instruction: Requires calculations similar to Exercise 1:

$$f = \frac{ax + b}{cx - d}$$

Với $a = 1, b = 2, c = 1, d = 2$

Hiện tượng gì sẽ xảy ra khi $x = 2$?

❖ Exercise 3 - If Else Expression: Input value a from the keyboard, calculate and print value c given $b = 20$, $d = 10$.

```
if (a >= 0) {  
    c = b + d;  
} else {  
    c = b - d;  
}
```

- ❖ Exercise 4 - For Loop: Use a for loop to output the value of the n-th Fibonacci number, where n is entered from the keyboard.

```
1  f(0) = 0;  
2  f(1) = 1;  
3  for( t0 = n, t0 >= 0, t0 --){  
4      f(t0) = f(t0-1) + f(t0-2)  
5  }
```

❖ Exercise 5 - Switch-case: Write the following switch-case statement in assembly language. Given $b = 10$, $c = 5$. The input value is entered from the keyboard.

```
switch (input)
{
    Case 0: a = b + c; break;
    case 1: a = b - c; break;
    case 2: a = b × c; break;
    case 3: a = b ÷ c; break;
    default: NOP; break;
}
```

- ❖ Exercise 6 - While Loop: Use a while loop to calculate the sum of the first n numbers. The value of n is entered from the keyboard and $n > 4$.

```
i    = 0;  
sum = 0;  
while ( i != n) {  
    sum = sum + i;  
    i   = i + 1;  
}
```

- ❖ Exercise 7 - Sorting Arrays: Given an array of 10 elements, write a program to sort it in ascending order.

1 1, 6, 3, 23, 3, 7, 1, 8, 34, 24, 50

❖ Exercise 8 - String Processing: Write a program to process the following string, converting uppercase characters to lowercase and vice versa. Note that this only applies to letters.

String: “Kien truc May tinh 2026”

❖ Creating and Using Subroutines

- Subroutines (also called procedures or functions):
 - Usually consist of a group of combined commands;
 - Are used through names and parameters.
- The significance of using subroutines:
 - Dividing functions makes the program clearer, easier to understand, and easier to maintain;
 - Subroutines are written once and can be used many times.

❖ Declaring a subroutine:

```
1  <proc_name> PROC
2
3      ; here goes the code
4      ; of the procedure ...
5
6      RET
7  <proc_name> ENDP
```

❖ Using a subroutine:

```
1
2  CALL <proc_name>
```

❖ Example: function m2 - multiplying 2

```
1  MOV AL, 1
2  MOV BL, 2
3
4  CALLL m2
5  ; Cac cau lenh khac
6  MOV CX, 30
7
8  ; Dinh nghia mot chuong trinh con
9  ; Input: AL, BL
10 ; Output: AX
11 m2 PROC
12     MUL BL    ; AX = AL * BL
13     RET      ; Tro ve chuong trinh chinh
14 m2 ENDP
```

❖ Parameter Passing in Subroutines

- Serves to exchange data between the calling program and the subroutine;
- Methods of parameter passing:
 - Passing parameters via registers
 - Loading values into registers to store the parameters to be passed before calling or returning from the subroutine.
 - Passing parameters via global variables
 - Global variables (defined in the data segment in the main program) can be accessed in both the main program and the subroutine.
 - Passing parameters via the stack
 - Using a combination of PUSH/POP instructions to pass parameters.

❖ Parameter Passing in Subroutines

■ Protecting Registers:

- It is necessary to protect the values of registers used in the calling program when they are also used in the subroutine.
- The values of registers can be changed in the subroutine → resulting in incorrect output in the calling program.

■ Methods for protecting registers:

- Use PUSH and POP for general, index, and pointer registers;
- Use PUSHF and POPF for flag registers;
- Use a unified convention for register usage.

- ❖ A macro is a named piece of code that can be inserted anywhere in a program's code.
 - Characteristics of macros:
 - Macros support parameter lists.
 - Macros only exist during code editing. When compiled, macros are replaced by their actual code.
 - If a macro is not used, its code is removed from the program after compilation.
 - Macros are faster than procedures/functions because their code is inserted directly into the program and they do not require call mechanisms (address saving) and return mechanisms (recovering the return address) like subroutines.

❖ Macro definition

```
1  name_macro MACRO [parameters,...]  
2      ;instructions  
3  ENDM
```

❖ Macro use

```
1  name_macro [real parameters,...]
```

❖ Example

```
1  MyMacro MACRO p1, p2, p3
2      MOV AX, p1
3      MOV BX, p2
4      MOV CX, p3
5  ENDM
6  ;...
7  MyMacro 1, 2, 3
8  MyMacro 4, 5, DX
```

After compiling:

```
1  MOV AX, 00001h
2  MOV BX, 00002h
3  MOV CX, 00003h
4  MOV AX, 00004h
5  MOV BX, 00005h
6  MOV CX, DX
```


Some illustrative examples

❖ Display English and Vietnamese greetings:

```

1      .Model Small
2      .Stack 100H
3      .Data
4          CRLF DB 13, 10, '$'
5          ChaoTay DB 'Hello!$'
6          ChaoTa DB 'Xin_lchao!$'
7      .Code
8      MAIN Proc
9          MOV AX, @Data; Khoi tao dau thanh ghi DS
10         MOV DS, AX
11
12         ; Hien thi loi chao Tay dung ham 9 cua ngat INT 21H
13
14         MOV AH, 9
15         LEA DX, ChaoTay
16         INT 21H
17
18         ; Xuong 5 dong dung ham 9 ngat INT 21H
19         LEA DX, CRLF
20         INT 21H
21         MOV CX, 6 ; CX chua so dong cach + 1
22     LAP:

```

❖ Display English and Vietnamese greetings:

```

23      INT 21H
24      LOOP LAP
25
26      ; Hien thi loi chao Ta dung ham 9 cua ngat INT 21H
27      MOV AH, 9
28      LEA DX, ChaoTa
29      INT 21H
30
31      ; Ket thuc tro ve chuong trinh goi
32      ; Dung ham 4CH cua ngat 21H
33      MOV AH, 4CH
34      INT 21H
35
36      MAIN Endp
37      END MAIN
  
```

❖ Change lowercase characters in a string to uppercase

```

1      .Model Small
2      .Stack 100H
3      .Data
4          ; Source string
5      str1 DB 'a','5', 'B', '?', 'd', 'g', 'P','N','k','*'
6          DB 10,13,'$';
7          ; Destination string
8      str2 DB 10 DUP('')
9          DB '$'
10     .Code
11     MAIN Proc
12         MOV AX, @Data; Khởi tạo đầu thanh ghi DS
13         MOV DS, AX
14         MOV ES, AX
15         ; Thiết lập SI trỏ tới str1 và DI trỏ tới str2
16         LEA SI, str1
17         LEA DI, str2
18         CLD
19         MOV CX, 10
20     START:
21         LODSB
22         ; Kiểm tra có phải chữ hoa không

```

❖ Change lowercase characters in a string to uppercase

```

23      CMP AL, 'a'
24      JL NotLowerCase
25      CMP AL, 'z'
26      JG NotLowerCase
27      ; Neu la chu viet thuong chuyen thanh chu viet hoa
28      SUB AL, 20H
29      ;Luu vao string moi
30      NotLowerCase:
31      STOSB
32      LOOP START
33      LEA DX, str1; In ra chuoi goc
34      MOV AH, 9
35      INT 21H
36      LEA DX, str2; In ra chuoi da bien doi
37      MOV AH, 9
38      INT 21H
39      ; Ket thuc tro ve chuong trinh Dung ham 4CH cua ngat 21H
40      MOV AH, 4CH
41      INT 21H
42      MAIN Endp
43      END MAIN

```

Some illustrative examples

❖ Find the largest number in a sequence

```

1      .Model Small
2      .Stack 100H
3      .Data
4          list DB 1,4,0,9,7,2,4,6,2,5
5      .Code
6      MAIN Proc
7          MOV AX, @Data; Khoi tao dau thanh ghi DS
8          MOV DS, AX
9
10         CLD
11         MOV CX, 9
12         LEA SI, list; SI tro den list
13         MOV BL, [SI]; Gan Max <-- phan tu dau tien
14         INC SI
15     START:
16         LODSB
17
18         CMP AL, BL
19         JLE BYPASS
20         MOV BL, AL; Neu AL>BL --> BL se duoc cap nhat gia tri
21
22     BYPASS:

```


❖ Find the largest number in a sequence

```

23      LOOP START
24
25      ; In ra gia tri lon nhat
26      ADD BL, '0'; bien doi tu so thanh ky tu
27      MOV DL, BL
28      MOV AH, 2
29      INT 21H
30
31      ; Ket thuc tro ve chuong trinh Dung ham 4CH cua ngat 21H
32      MOV AH, 4CH
33      INT 21H
34      MAIN Endp
35      END MAIN

```

Some illustrative examples

❖ Sort and print the number sequence in descending order

```

1      .Model Small
2      .Stack 100H
3      .Data
4          LIST_COUNT EQU 10
5          list DB 1,4,0,3,7,2,8
6          CRLF DB 13,10,'$'
7      .Code
8      Main PROC:
9          MOV AX, @Data; nap dia chi doan du lieu vao AX
10         MOV DS, AX; nap dia chi doan du lieu vao DS
11
12         ; In danh sach ban dau
13         MOV CX, LIST_COUNT
14         LEA SI, list
15         CALL printList
16         LEA SI, list
17         MOV BL, 1; Bien dem luu vao
18
19         MainLoop:
20             MOV AL, [SI]; AL <-- [SI]

```

Some illustrative examples

❖ Sort and print the number sequence in descending order

```

21         MOV DI, SI
22         MOV BH, BL; Bien dem phu
23         MOV DX, DI; DX luu vi tri cua gia tri MIN
24     SubLoop:
25         INC DI
26         INC BH
27         CMP , [DI]
28         JLE NotMin
29         MOV AL, [DI]
30         MOV DX, DI
31     NotMin:
32         CMP BH, LIST_COUNT
33         JE ExitSub
34         JMP SubLoop
35     ExitSub:
36         ; Hoan doi vi tri neu MIN khac voi vi tri dau tien
37         MOV DI, DX
38         CMP SI, DI
39         JE NoSwap
40         CALL swapMemLocation
41     NoSwap:
42         INC BL
43         CMP BL, LIST_COUNT

```


Some illustrative examples

❖ Sort and print the number sequence in descending order

```

44         JE ExitMain
45         INC SI
46         JMP MainLoop
47 ExitMain:
48         LEA DX, CR
49         CALL printString
50
51         MOV CX, LIST_COUNT
52         LEA SI, list
53         CALL printList
54         ; Ket thuc tro ve chuong trinh goi
55         ; Dung ham 4CH cua ngat 21H
56         MOV AH, 4CH
57         INT 21H
58 Main ENDP
59         ; Cac chuong trinh con
60
61         ; Chuong trinh con hoan doi hai vi tri
62         ; Input: SI tro toi vi tri thu nhat
63         ;      DI tro toi vi tri thu hai
64 swapMemLocation PROC
65         PUSH AX,
66         MOV AL, [SI]

```

Some illustrative examples

❖ Sort and print the number sequence in descending order

```

67      MOV AH, [DI]
68      MOV [SI], AH
69      MOV [DI], AL
70      POP AX
71      RET
72      swapMemLocation ENDP
73
74      ; Chuong trinh con in ra danh sach - list
75      ; Input: SI luu dia chi bat dau cua danh sach
76      ;      CX luu tru so luong phan tu cua danh sach
77      printList PROC
78          PUSH DX
79          StartPrint:
80              MOV DL, [SI]
81              CALL printSingleDigital
82              INC SI
83              LOOP StartPrint
84              POP DX
85              RET
86      printList ENDP
87
88      ; Chuong trinh con in ra chuoai ket thuc boi $
89      ; Input: DX tro toi chuoai can in

```

Some illustrative examples

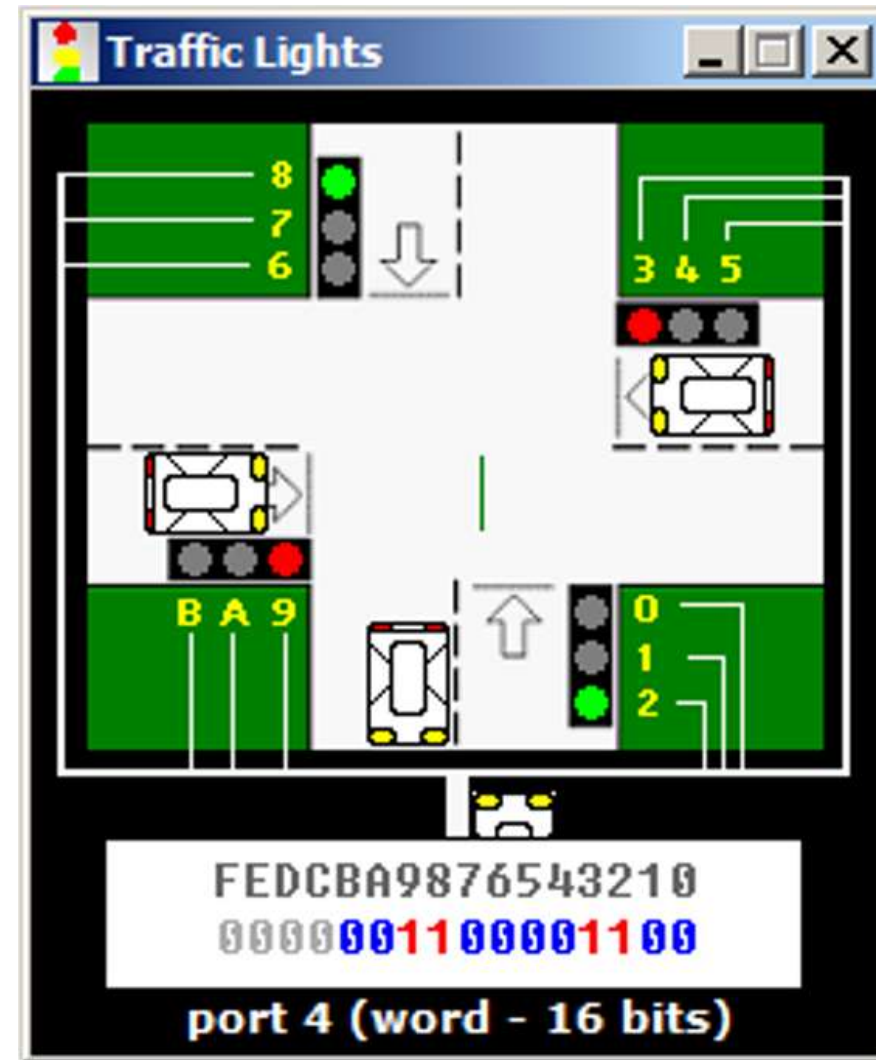
❖ Sort and print the number sequence in descending order

```
90      printString PROC
91          PUSH AX
92          MOV AH, 9
93          INT 21H
94          POP AX
95          RET
96      printString ENDP
97
98      ; Chuong trinh con in Mot ky tu so
99      ; Input: DL chua ky tu so can in
100     printSingleDigit PROC
101         PUSH AX,
102         ADD DL, '0'
103         MOV AH, 2
104         INT 21H
105         POP AX
106         RET
107     printSingleDigit ENDP
108
109     END Main
```

❖ Traffic Light Simulation Device

Simulation - Traffic Lights:

- The virtual traffic light system uses port 4 – a 16-bit port – to receive control information;
- Use 12 bits (0-11) for 4 groups of lights:
 - Each group consists of 3 lights: Green, Yellow, and Red;
 - Bit 0 – turn off the light, bit 1 – turn on the light
- 4 bits (12-15) are not used – should be set to 0.



❖ Traffic Light Simulation Device Simulation - Traffic Lights:

■ Traffic light control:

- Send control word (2 bytes) to port 4;
- The bits of the control word are set to match the intended traffic light control (Bit 0 – turn off the light, bit 1 – turn on the light)
- Example: control bits 0000 001 100 001 100

GYR GYR GYR GYR

- Use function 86h of BIOS interrupt 15h to create a timeout – the time to hold the newly set state of the traffic light cluster
- The number of microseconds is set in CX:DX before calling the interrupt.

❖ Traffic Light Simulation Device Simulation - Traffic Lights:

```

1  .Model Small
2  .Stack 100H
3  .Data
4      ; 0000 GYR GYR GYR GYR
5      R1 DW 0000 0011 0000 1100B; Di doc
6      R2 DW 0000 0010 1000 1010B; Doc --> vang
7      R3 DW 0000 1000 0110 0001B; Di ngang
8      R4 DW 0000 0100 0101 0001B; ngang --> vang
9      ; FEDC BA9 876 543 210
10
11     ALL_RED EQU 0000 0010 0100 1001B
12     PORT EQU 4; Output port
13
14     ; Hang so thoi gian (s)
15     ; 3s = 3,000,000 = 002D C6C0H
16     WAIT_3_SEC_CX EQU 2Dh
17     WAIT_3_SEC_DX EQU 0C6C0h
18
19     ; 10s = 10,000,000 = 0098 9680H
20     WAIT_10_SEC_CX EQU 98h
21     WAIT_10_SEC_DX EQU 9680h

```

❖ Traffic Light Simulation Device Simulation - Traffic Lights:

```

22      .Code
23      ; Dinh nghi mot Macro
24      waitMacro MACRO t1, t2
25          MOV CX, t1
26          MOV DX, t2
27          MOV AH, 86H
28          INT 15H
29      ENDM
30      MAIN Proc
31      ; Khoi tao cho thanh ghi DS
32      MOV AX, @Data; nap dia chi doan du lieu vao AX
33      MOV DS, AX; nap dia chi doan du lieu vao DS
34
35      ; Dat den do cho cho tat ca cac huong
36      ; Set lights to Red for all direction
37
38      MOV AX, ALL_RED
39      OUT PORT, AX
40      waitMacro WAIT_3_SEC_CX, WAIT_3_SEC_DX
41      Start:
42      LEA SI, R1
43      MOV AX, [SI]
44      OUT PORT, AX
    
```

❖ Traffic Light Simulation Device Simulation - Traffic Lights:

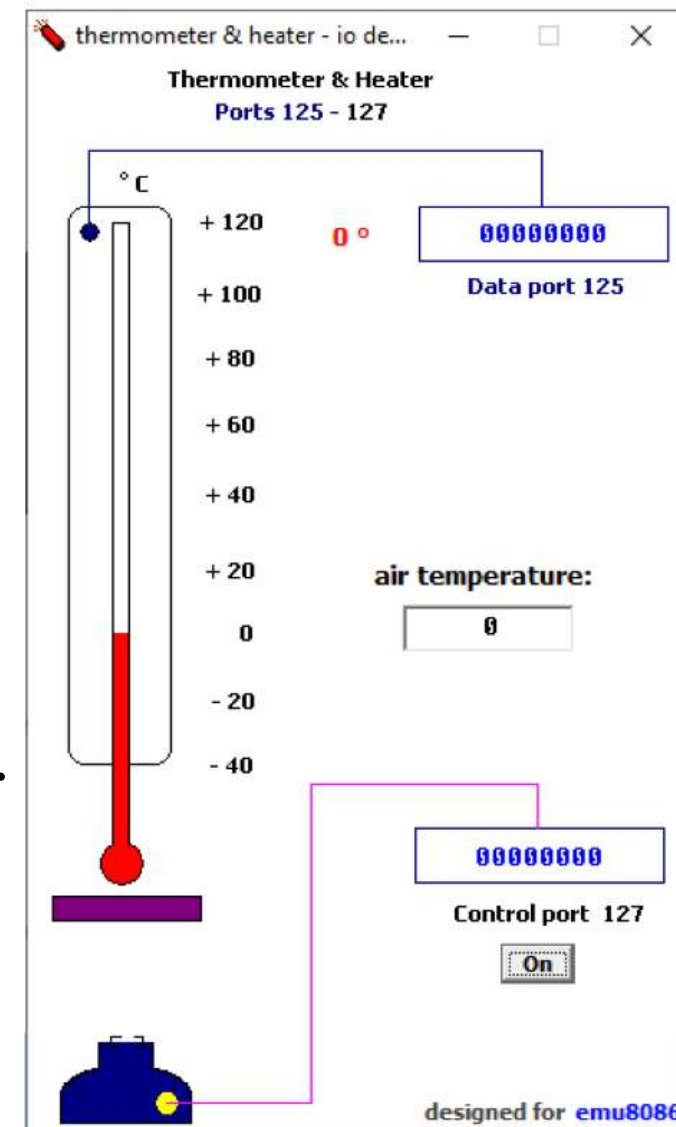
```

45
46      waitMacro WAIT_10_SEC_CX, WAIT_10_SEC_DX
47      LEA SI, R2
48      MOV AX, [SI]
49      OUT PORT, AX
50      waitMacro WAIT_3_SEC_CX, WAIT_3_SEC_DX
51      LEA SI, R3
52      MOV AX, [SI]
53      OUT PORT, AX
54      waitMacro WAIT_10_SEC_CX, WAIT_10_SEC_DX
55      LEA SI, R4
56      MOV AX, [SI]
57      OUT PORT, AX
58      waitMacro WAIT_3_SEC_CX, WAIT_3_SEC_DX
59      JMP Start
60      ; Ket thuc tro ve chuong trinh goi
61      ; dung ham 4CH cua ngat 21H
62      MOV AH, 4CH
63      INT 21H
64      MAIN Endp
65      END MAIN

```


❖ Virtual Thermometer & Heater Device:

- Uses 2 ports:
 - Digital control port 127 to receive control bytes for the heater:
 - Send 0 to turn off the heater
 - Send 1 to turn on the heater.
 - Digital data port 125 to read the temperature data from the thermometer.



❖ Characteristics

```

1      .Model Small
2      .Stack 100H
3      .Data
4          ; Khai bao cac bien va hang
5      .Code
6      MAIN Proc
7          ; Khoi tao cho thanh ghi DS
8      MOV AX, @Data; nap dia chi doan du lieu vao AX
9      MOV DS, AX; nap dia chi doan du lieu vao DS
10
11         ;Doc nhiet do hien tai
12     IN AL, 125
13
14     CMP AL, 60
15     JL low ; Nhay den low neu AL nho hon 60
16
17     CMP AL, 80
18     JLE ok; Nhay den ok neu AL nho hon hoac bang 80
19     JG high; Nhay den high neu AL lon hon 80
20
21     low:

```

❖ Characteristics

```

22         MOV AL, 1
23         OUT 127, AL; Bat bep "ON"
24         JMP ok
25     high:
26         MOV AL, 0
27         OUT 127, AL; Tat bep "OFF"
28
29     ok:
30         JMP Start
31
32     ; Ket thuc tro ve chuong trinh goi
33     ; dung ham 4CH cua ngat 21H
34     MOV AH, 4CH
35     INT 21H
36     MAIN Endp
37     END MAIN
    
```

❖ Exercise: Draw a flowchart and write a program to control a stove so that its temperature remains stable within the range of 70-100°C. The system is connected to an 8086 microcontroller, where: the temperature reading port is 100, the temperature value is an 8-bit signed number corresponding to the actual temperature. The stove control port is 105H; when a value of 0 is output to the port, the stove turns off, and when a value of 1 is output, the stove turns on.